US 20210397414A1

(19) **United States**

(12) **Patent Application Publication** (10) **Pub. No.: US 2021/0397414 A1**

Raha et al. (43) **Pub. Date: Dec. 23, 2021**

(54) **AREA AND ENERGY EFFICIENT MULTI-PRECISION MULTIPLY-ACCUMULATE UNIT-BASED PROCESSOR**

(71) Applicant: **Intel Corporation**, Santa Clara, CA (US)

(72) Inventors: **Arnab Raha**, Portland, OR (US); **Mark A. Anders**, Hillsboro, OR (US); **Martin Power**, Chapelizod (IE); **Martin Langhammer**, Alderbury (GB); **Himanshu Kaul**, Portland, OR (US); **Debabrata Mohapatra**, Santa Clara, CA (US); **Gautham Chinya**, Sunnyvale, CA (US); **Cormac Brick**, San Francisco, CA (US); **Ram Krishnamurthy**, Portland, OR (US)

(21) Appl. No.: **17/358,868**

(22) Filed: **Jun. 25, 2021**

Publication Classification

(51) **Int. Cl.**

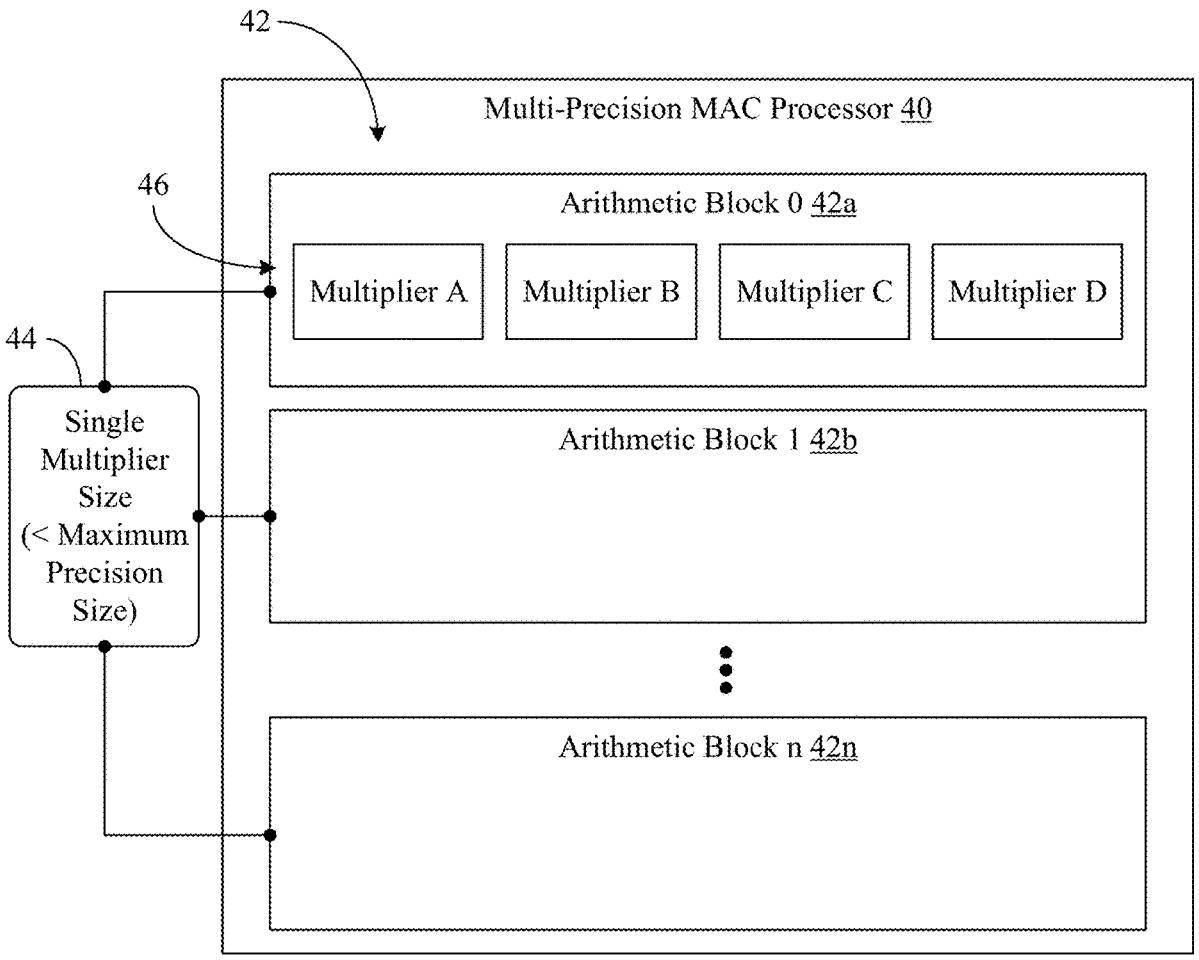
G06F 7/544	(2006.01)
G06F 7/527	(2006.01)
G06F 5/01	(2006.01)

(52) **U.S. Cl.**

CPC ***G06F 7/5443*** (2013.01); ***G06F 5/01*** (2013.01); ***G06F 7/5272*** (2013.01)

(57) **ABSTRACT**

Systems, apparatuses and methods may provide for multi-precision multiply-accumulate (MAC) technology that includes a plurality of arithmetic blocks, wherein the plurality of arithmetic blocks each contain multiple multipliers, and wherein the logic is to combine multipliers one or more of within each arithmetic block or across multiple arithmetic blocks. In one example, one or more intermediate multipliers are of a size that is less than precisions supported by arithmetic blocks containing the one or more intermediate multipliers.



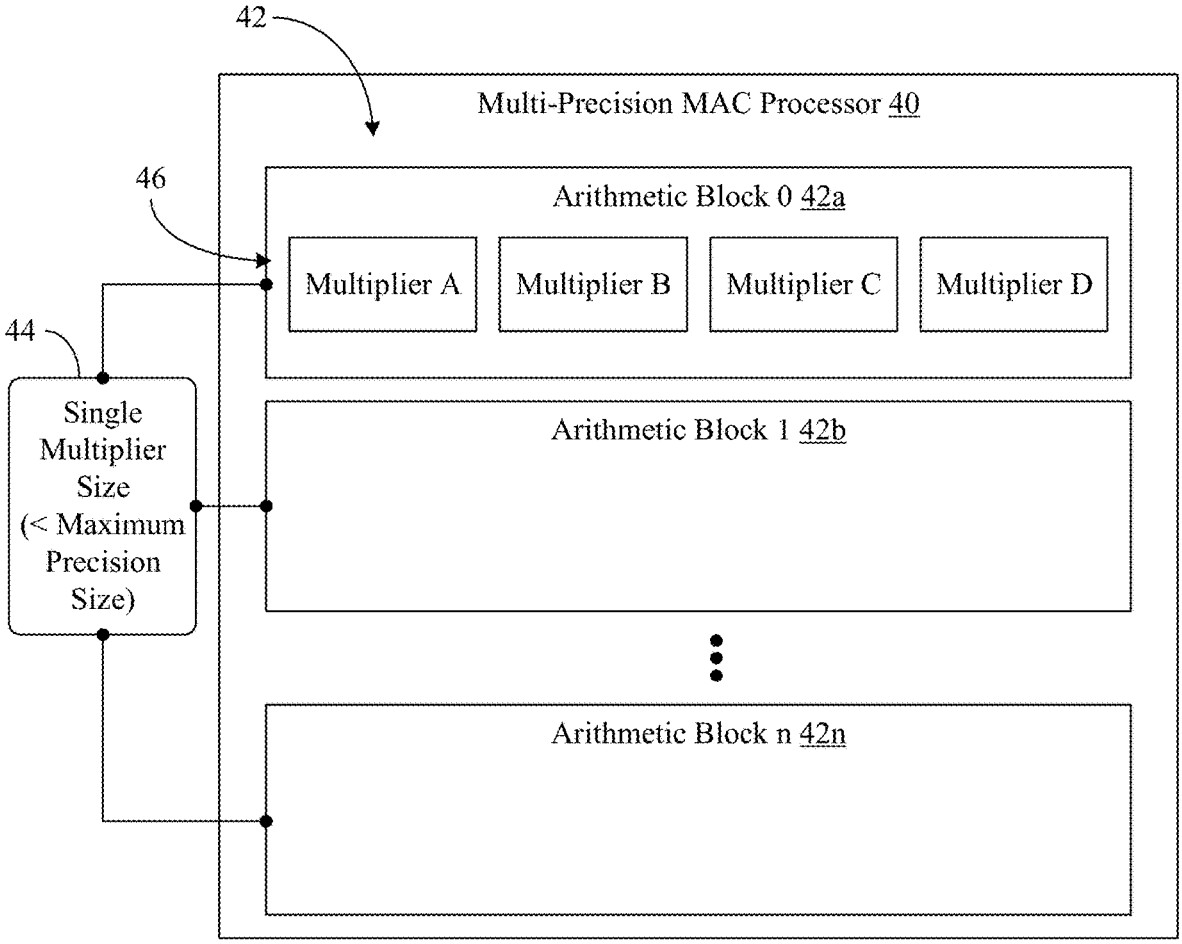

42
Multi-Precision MAC Processor 40
46
Arithmetic Block 0 42a
Multiplier A
Multiplier B
Multiplier C
Multiplier D
44
Single Multiplier Size (< Maximum Precision Size)
Arithmetic Block 1 42b
Arithmetic Block n 42n

FIG. 1

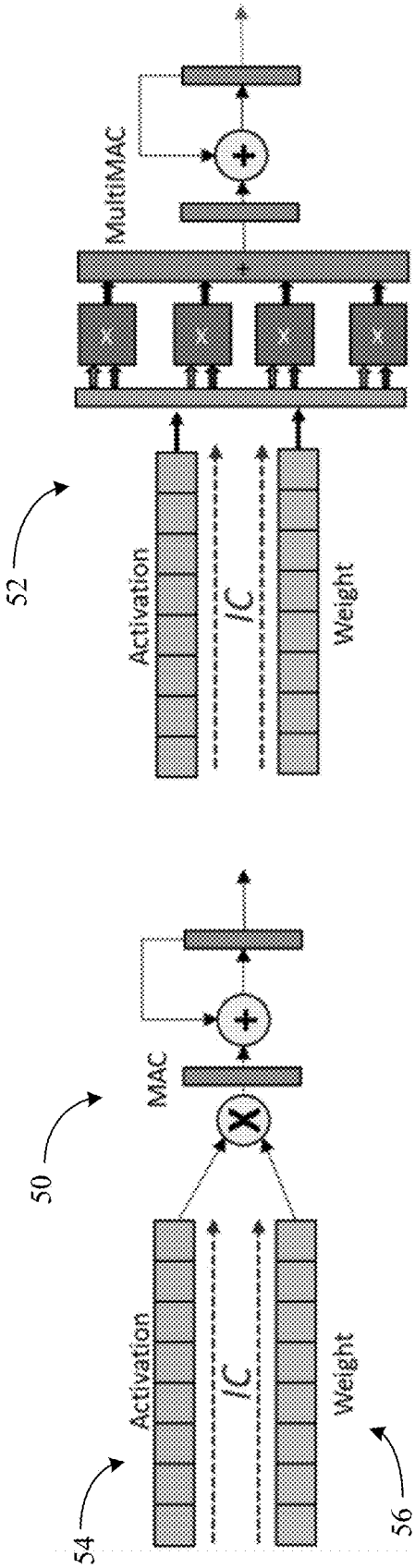

MultiMAC
52
Activation
IC
Weight
MAC
50
Activation
IC
Weight
54
56

FIG. 2

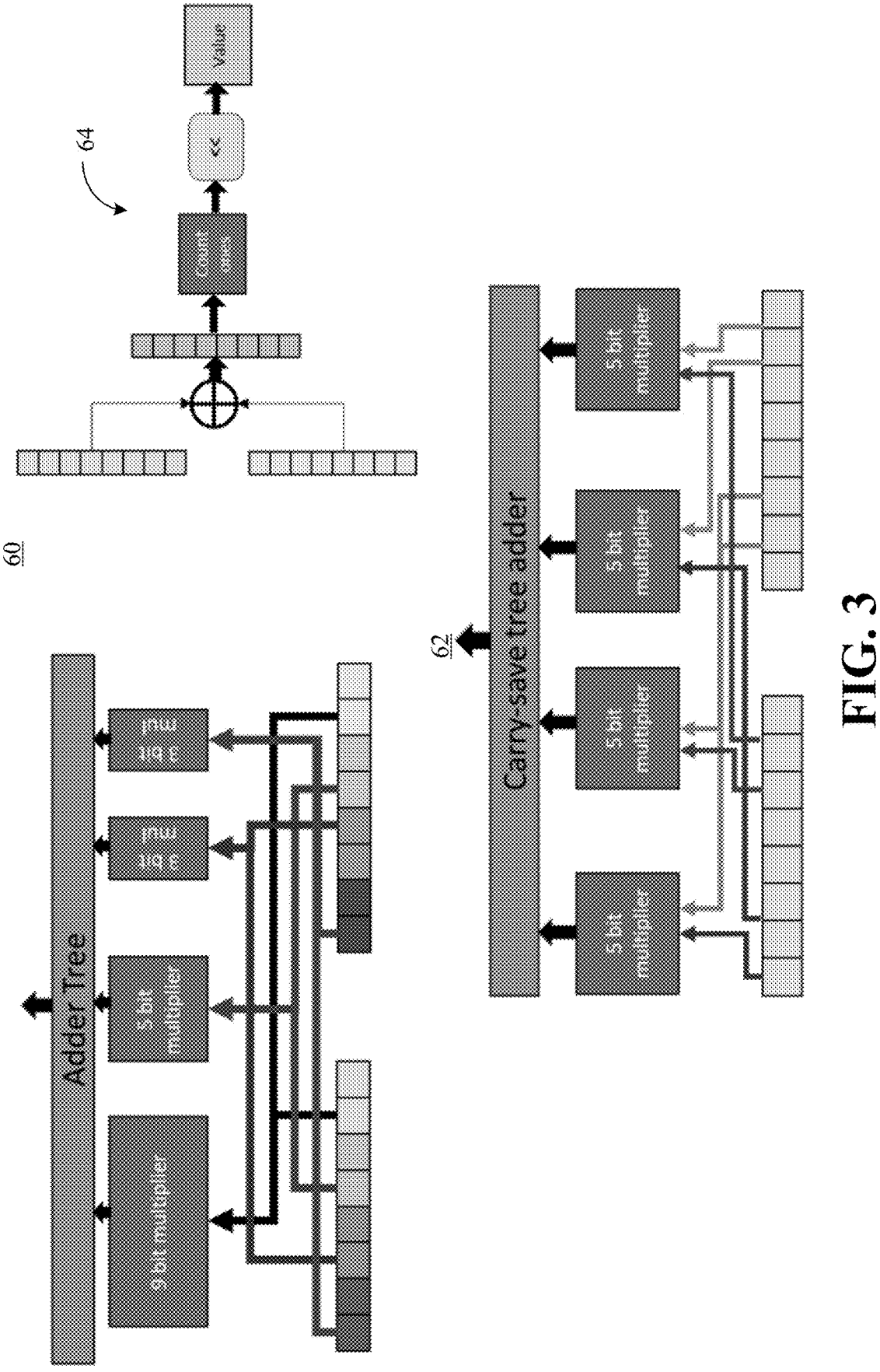

60
64
Value
<<
Count
62
Carry-save tree adder
3-bit multiplier
3-bit multiplier
3-bit multiplier
3-bit multiplier
Adder Tree
3-bit multiplier
3-bit multiplier
3-bit multiplier
3-bit multiplier

FIG. 3

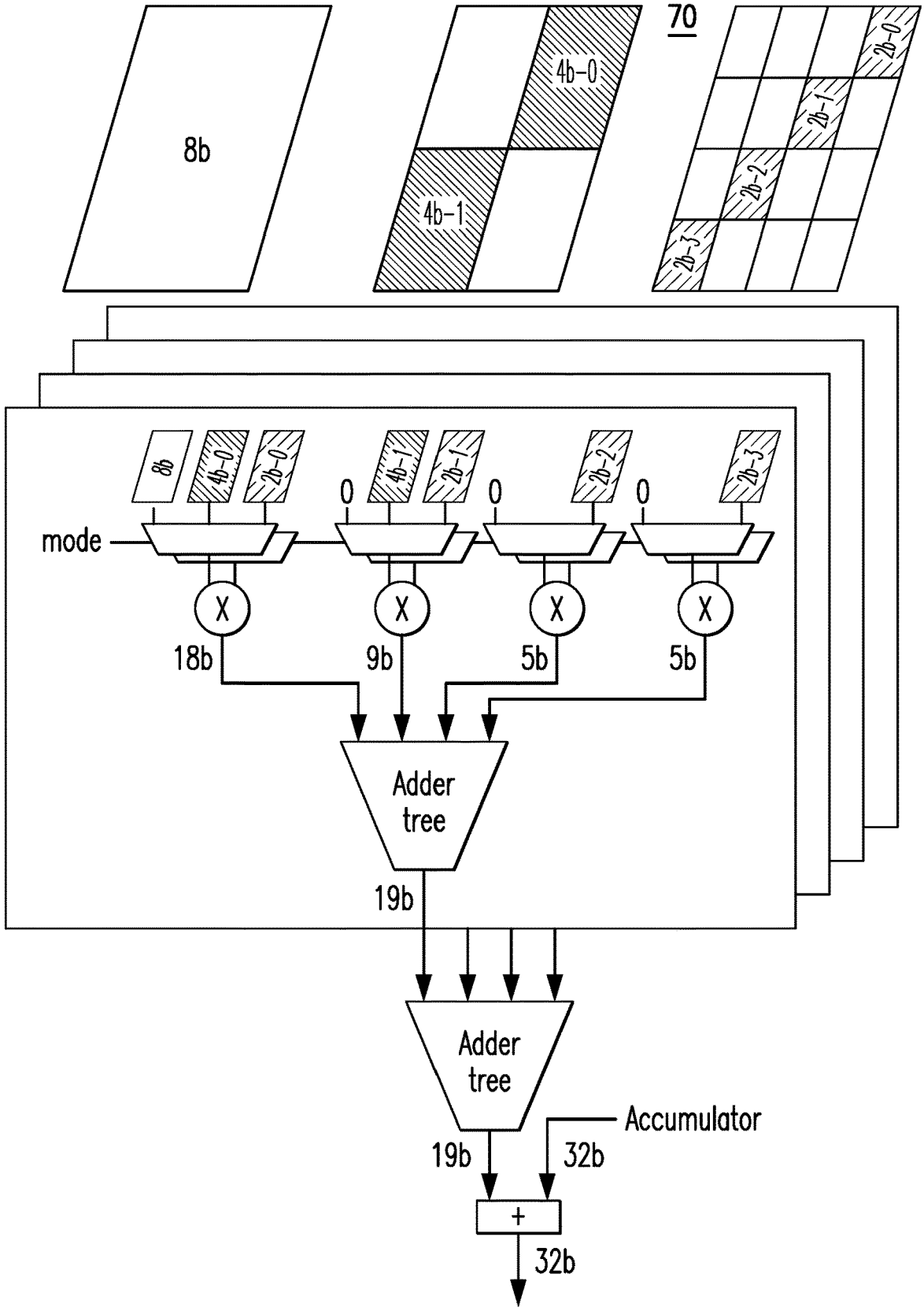

8b
4b-0
4b-1
70
2b-0
2b-1
2b-2
2b-3
8b
4b-0
2b-0
0
4b-1
2b-1
0
2b-2
0
2b-3
mode
X
X
X
X
18b
9b
5b
5b
Adder tree
19b
Adder tree
Accumulator
19b
32b
+
32b

FIG. 4

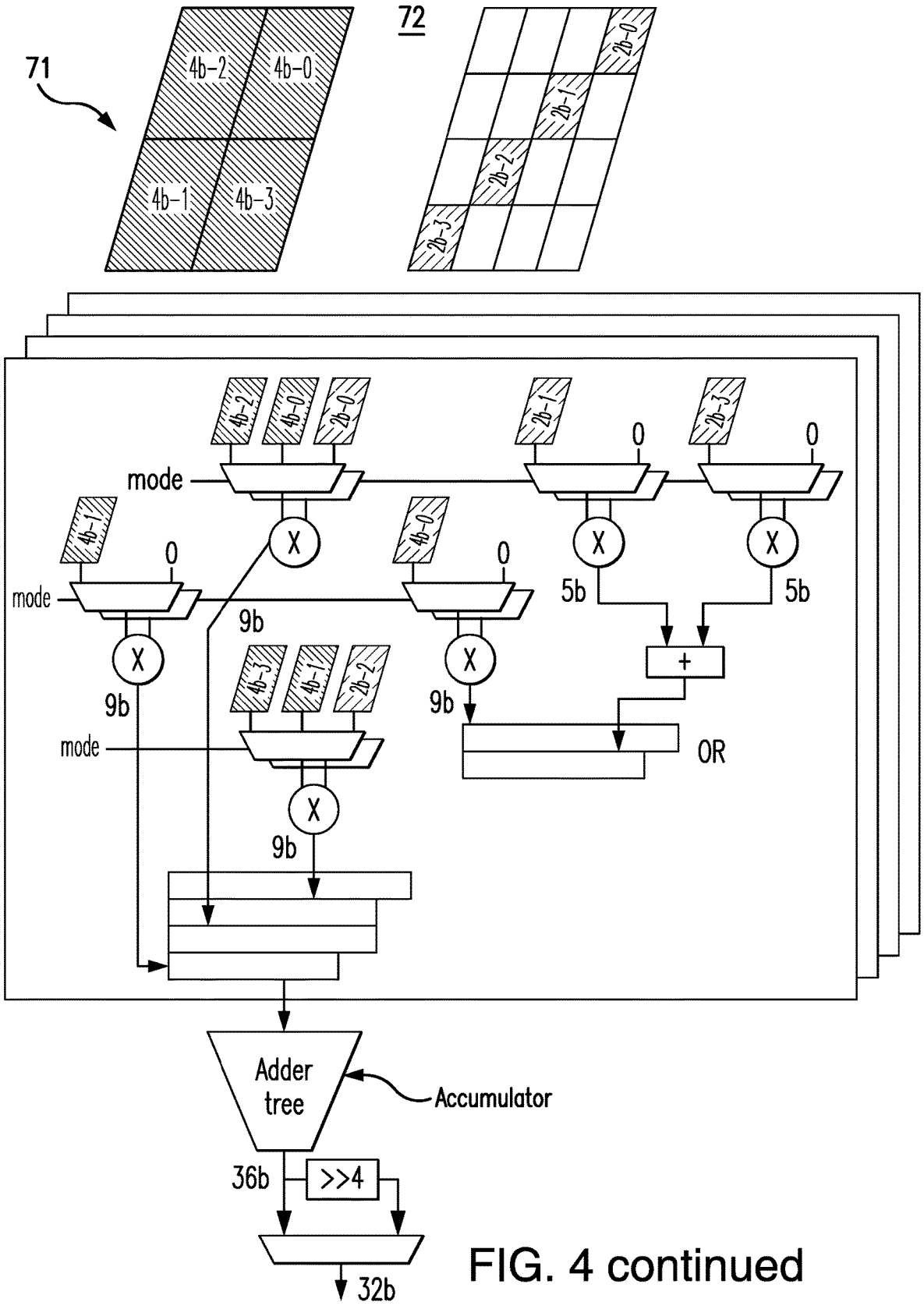


FIG. 4 continued

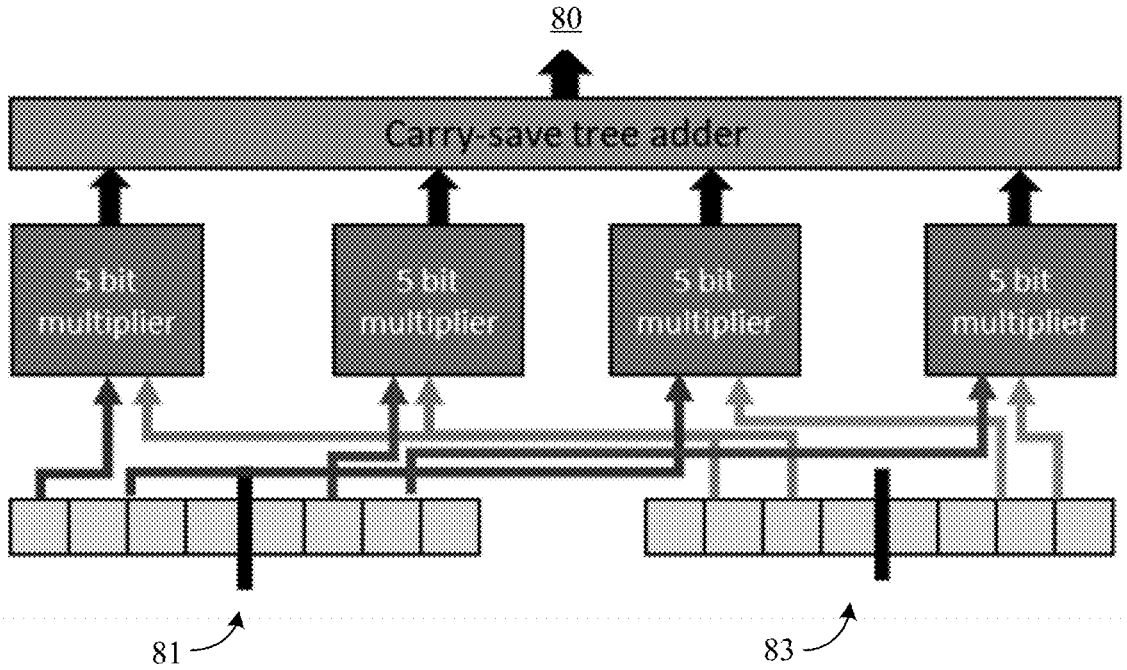

80
Carry-save tree adder
5 bit multiplier
5 bit multiplier
5 bit multiplier
5 bit multiplier
81
83

FIG. 5A

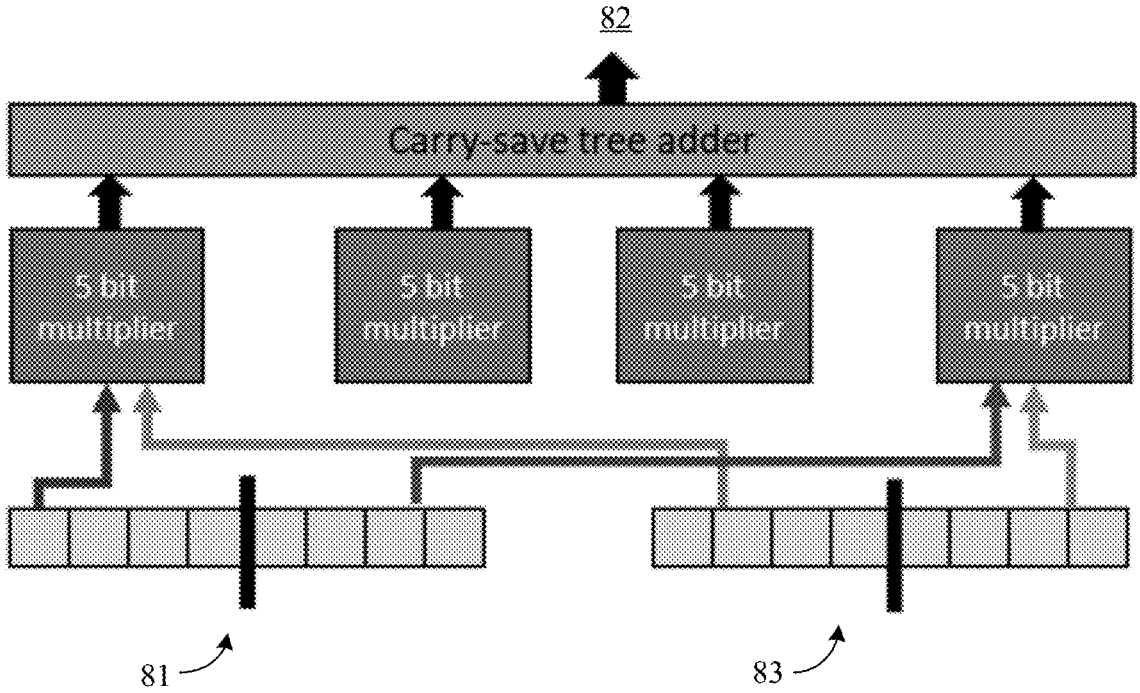

82
Carry-save tree adder
5 bit multiplier
5 bit multiplier
5 bit multiplier
5 bit multiplier
81
83

FIG. 5B

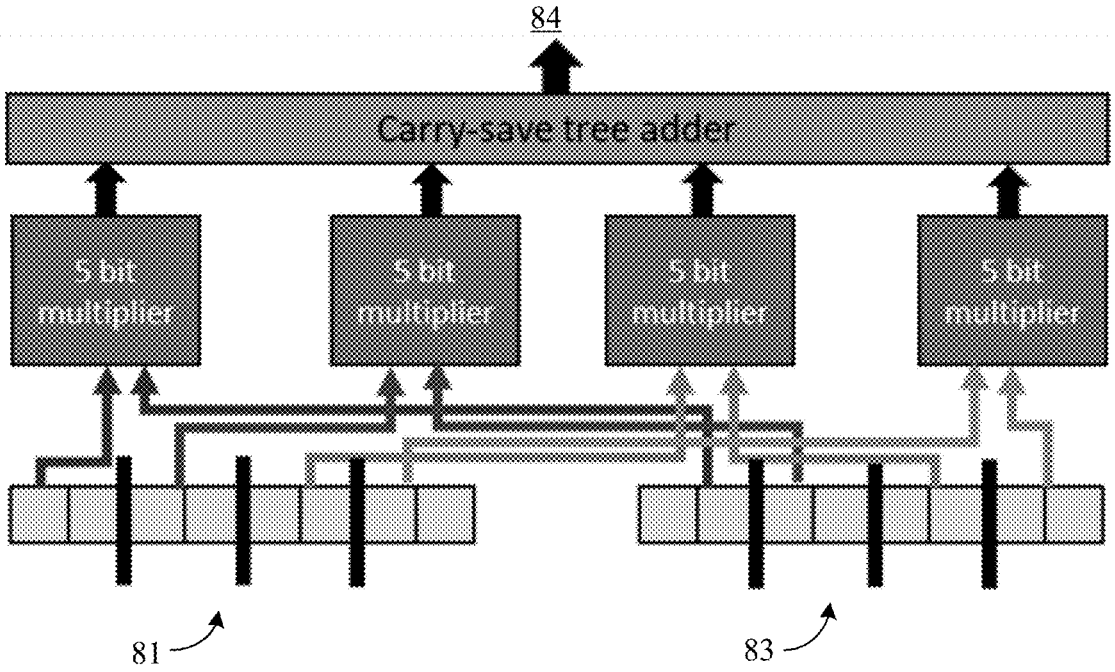

84
Carry-save tree adder
5 bit multiplier
5 bit multiplier
5 bit multiplier
5 bit multiplier
81
83

FIG. 5C

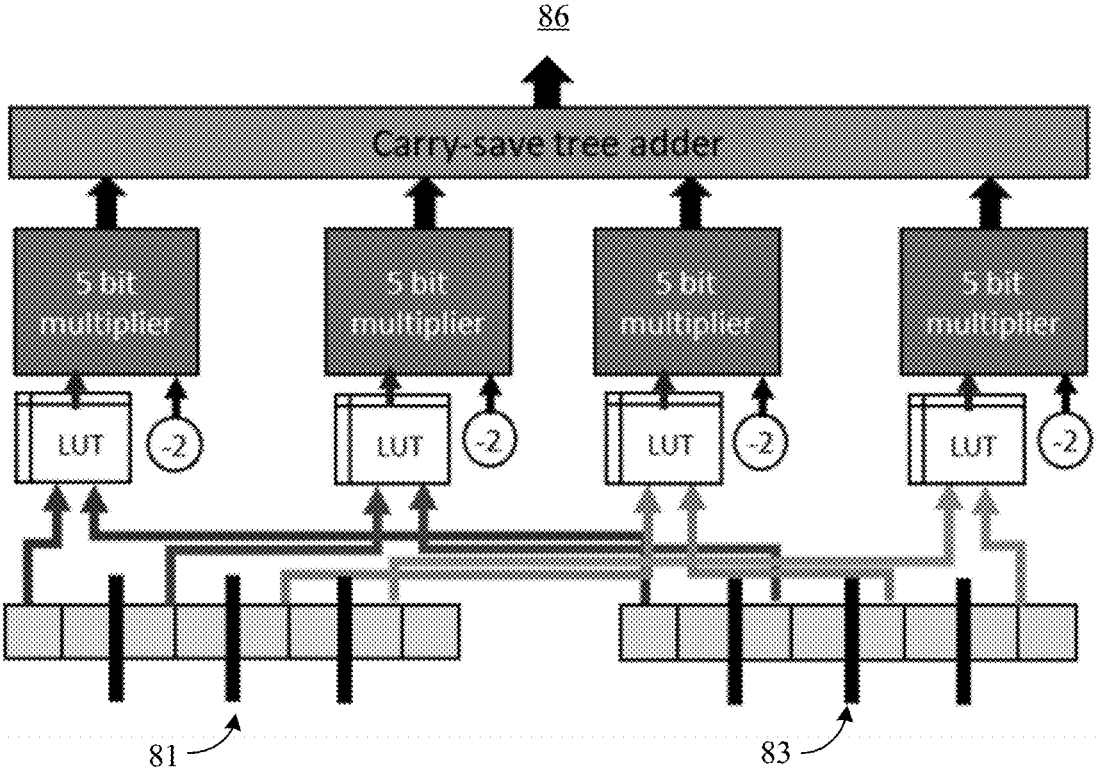
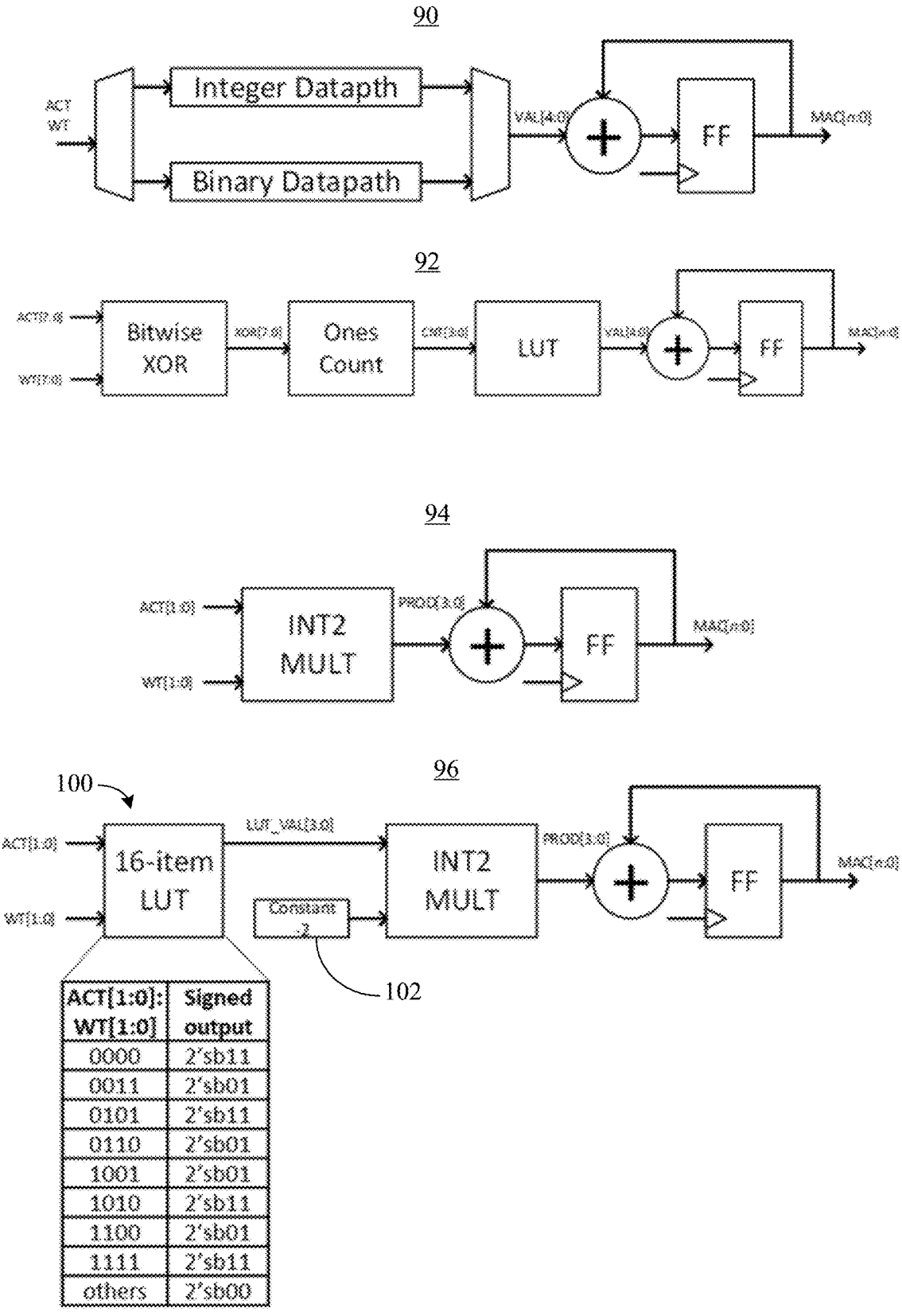

86
Carry-save tree adder
5 bit multiplier
5 bit multiplier
5 bit multiplier
5 bit multiplier
LUT
-2
LUT
-2
LUT
-2
LUT
-2
81
83

FIG. 5D



ACT[1:0]: WT[1:0]	Signed output
0000	2'sb11
0011	2'sb01
0101	2'sb11
0110	2'sb01
1001	2'sb01
1010	2'sb11
1100	2'sb01
1111	2'sb11
others	2'sb00

FIG. 6

98

ACT[1]	ACT[0]	WT[1]	WT[0]		ACT[1]	ACT[0]	WT[1]	WT[0]		PROD[1]	PROD[0]	SUM		MULT	CONST	RESULT
0	0	0	0		−1	−1	−1	−1		1	1	2		−1	−2	2
0	0	0	1		−1	−1	−1	1		1	−1	0		0	−2	0
0	0	1	0		−1	−1	1	−1		−1	1	0		0	−2	0
0	0	1	1		−1	−1	1	1		−1	−1	−2		1	−2	−2
0	1	0	0		−1	1	−1	−1		1	−1	0		0	−2	0
0	1	0	1		−1	1	−1	1		1	1	2		−1	−2	2
0	1	1	0		−1	1	1	−1		−1	−1	−2		1	−2	−2
0	1	1	1		−1	1	1	1		−1	1	0		0	−2	0
1	0	0	0		1	−1	−1	−1		−1	1	0		0	−2	0
1	0	0	1		1	−1	−1	1		−1	−1	−2		1	−2	−2
1	0	1	0		1	−1	1	−1		1	1	2		−1	−2	2
1	0	1	1		1	−1	1	1		1	−1	0		0	−2	0
1	1	0	0		1	1	−1	−1		−1	−1	−2		1	−2	−2
1	1	0	1		1	1	−1	1		−1	1	0		0	−2	0
1	1	1	0		1	1	1	−1		1	−1	0		0	−2	0
1	1	1	1		1	1	1	1		1	1	2		−1	−2	2

ACTs and WTs encoded as 0s and 1s

ACTs and WTs actual values as +1/−1

PROD[1]=ACT[1]*WT[1]
PROD[0]=ACT[0]*WT[0]
SUM=PROD[0]+PROD[1]

MULT=f(ACT[1:0],WT[1:0])
CONST=Constant
RESULT=MULT*CONST
SUM==RESULT

FIG. 7

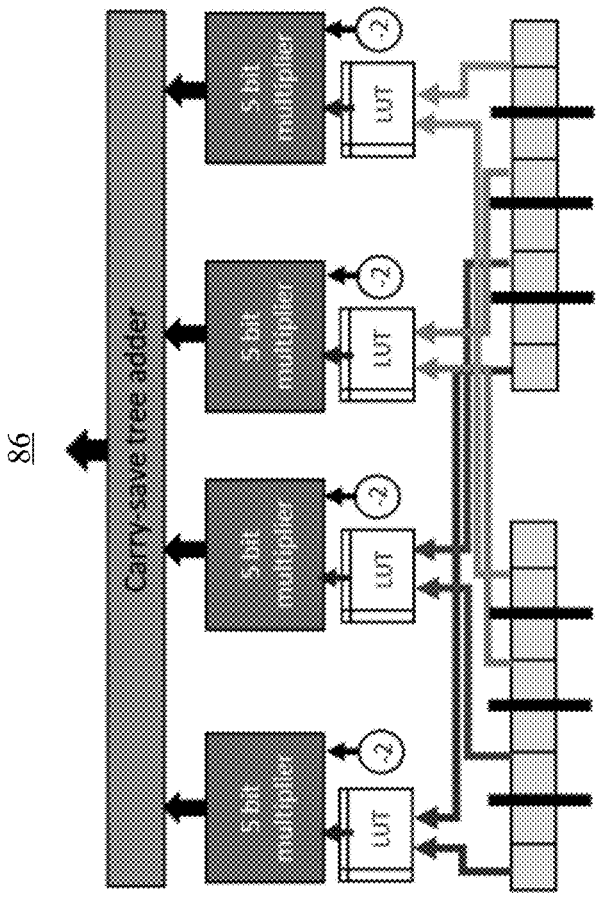
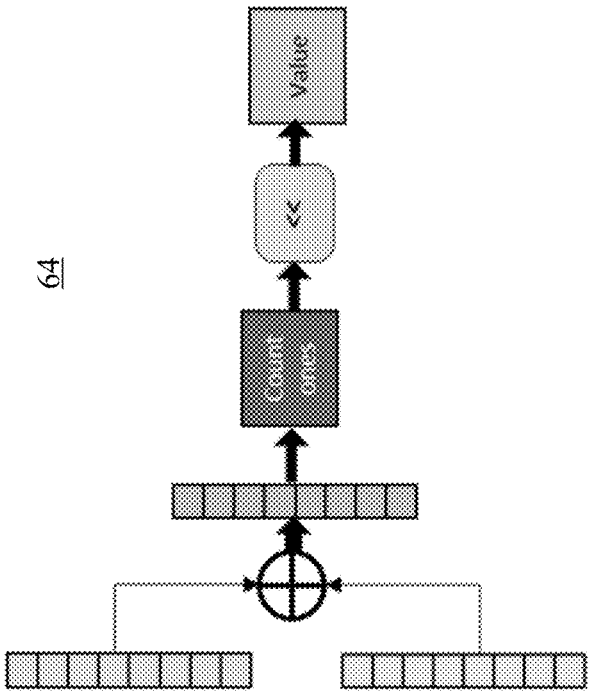

86
Carry-save tree adder
5:3 multiplier
LUT
?

FIG. 8


64
Value
<<
Output regs

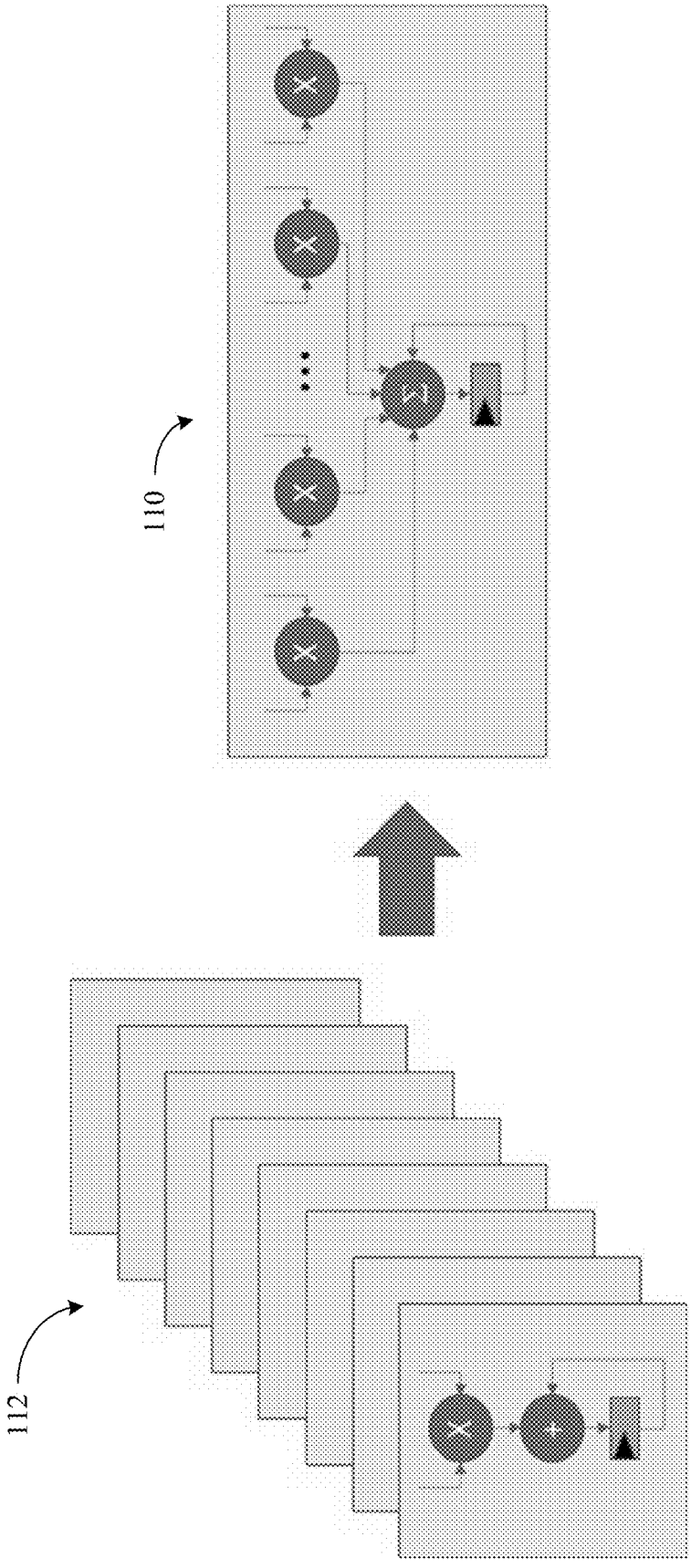


FIG. 9

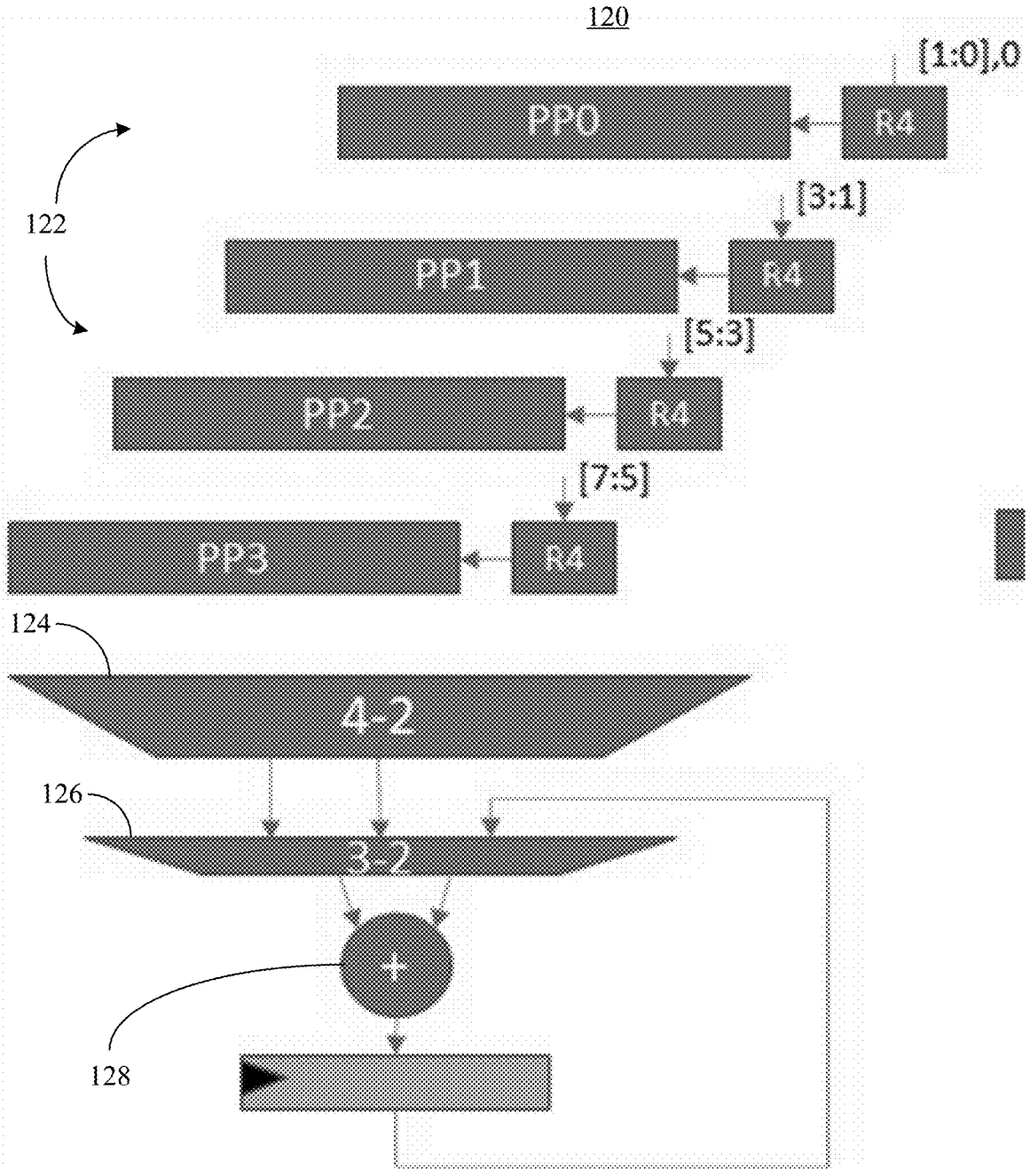


FIG. 10

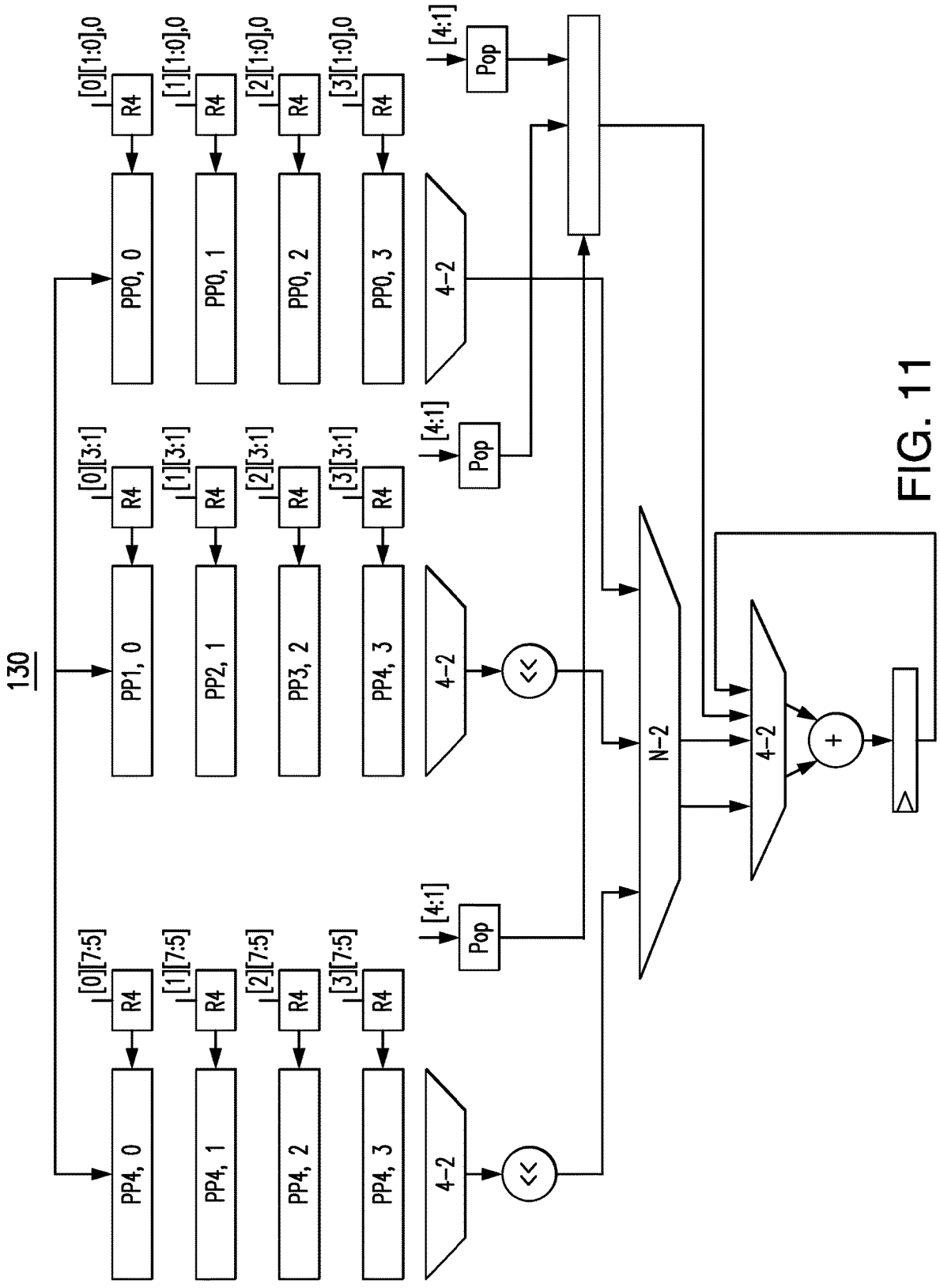

130
[0][1:0],0
[1][1:0],0
[2][1:0],0
[3][1:0],0
[4:1]
Pop
R4
PP0, 0
PP0, 1
PP0, 2
PP0, 3
4–2
[0][3:1]
[1][3:1]
[2][3:1]
[3][3:1]
PP1, 0
PP2, 1
PP3, 2
PP4, 3
<<
[0][7:5]
[1][7:5]
[2][7:5]
[3][7:5]
PP4, 0
PP4, 1
PP4, 2
PP4, 3
N–2
+

FIG. 11

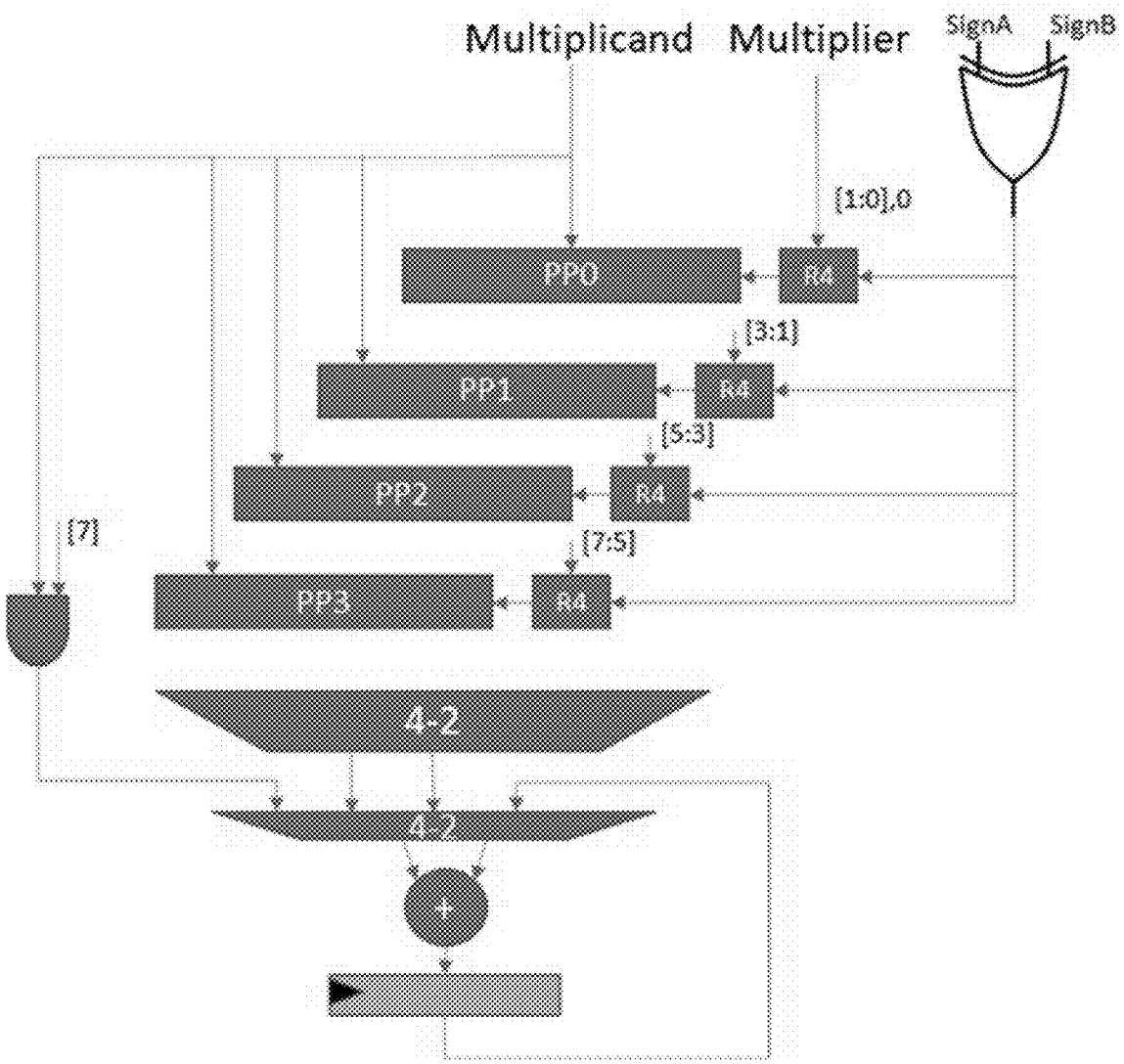

140
Multiplicand
Multiplier
SignA
SignB
{1:0},0
PP0
R4
{3:1}
PP1
R4
{5:3}
PP2
R4
{7}
{7:5}
PP3
R4
4-2
4-2
+

FIG. 12

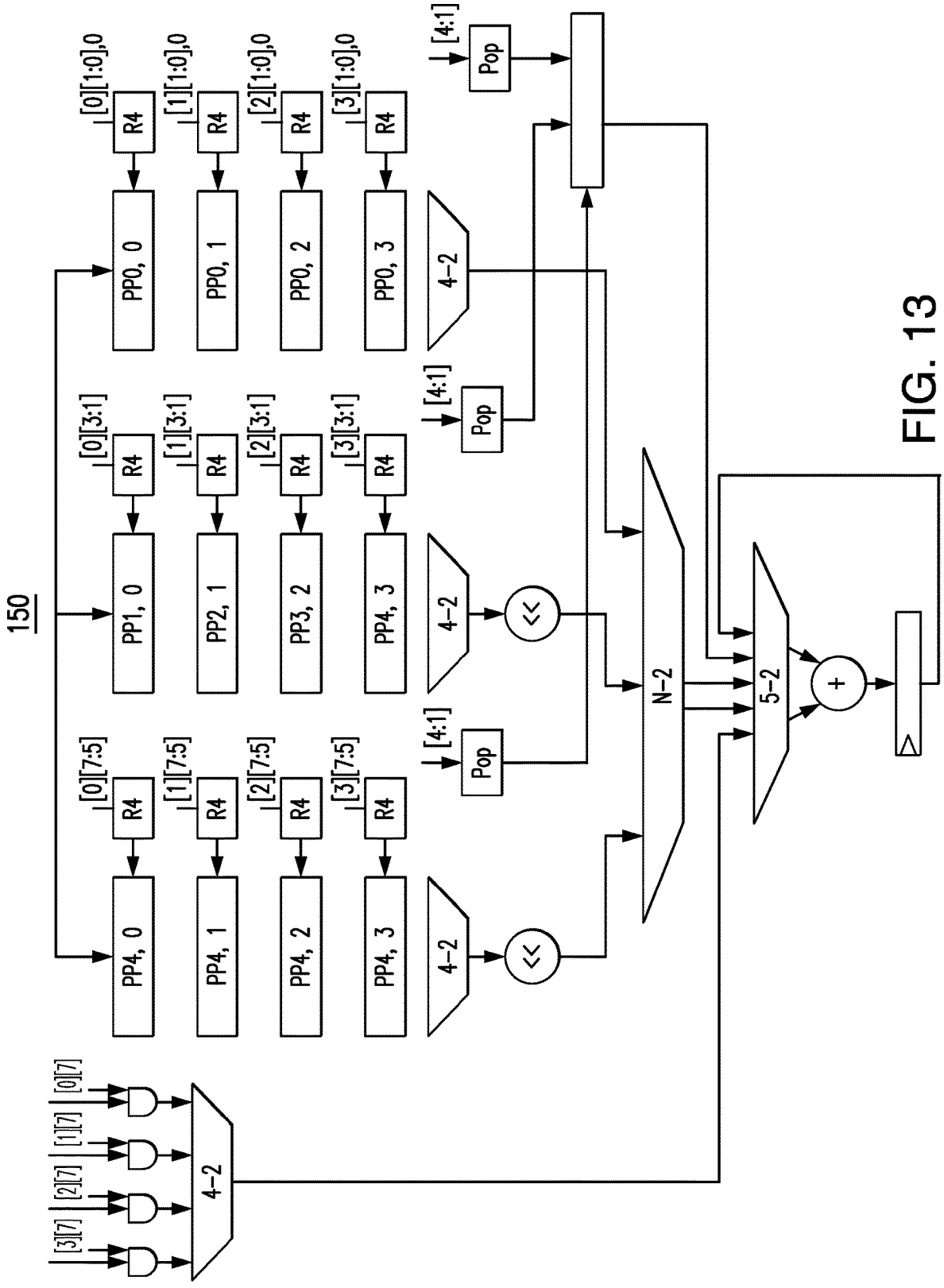

150
[3][7]
[2][7]
[1][7]
[0][7]
4–2
PP4, 0
PP4, 1
PP4, 2
PP4, 3
[0][7:5]
[1][7:5]
[2][7:5]
[3][7:5]
R4
4–2
[4:1]
Pop
PP1, 0
PP2, 1
PP3, 2
PP4, 3
[0][3:1]
[1][3:1]
[2][3:1]
[3][3:1]
R4
4–2
[4:1]
Pop
PP0, 0
PP0, 1
PP0, 2
PP0, 3
[0][1:0],0
[1][1:0],0
[2][1:0],0
[3][1:0],0
R4
4–2
[4:1]
Pop
N–2
5–2
+

FIG. 13

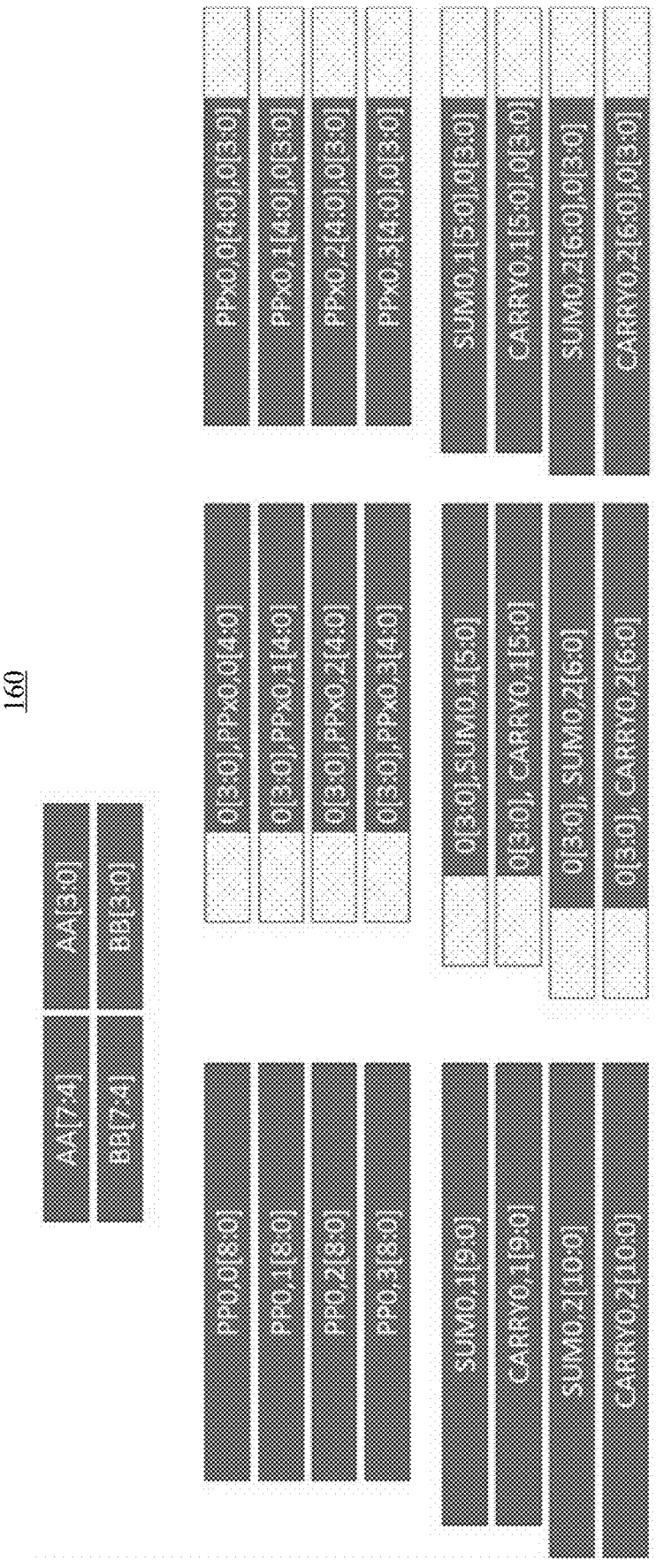

160
AA[7:4]
BB[7:4]
AA[3:0]
BB[3:0]
PP0.0[8:0]
PP0.1[8:0]
PP0.2[8:0]
PP0.3[8:0]
0[3:0]PP0.0[4:0]
0[3:0]PP0.1[4:0]
0[3:0]PP0.2[4:0]
0[3:0]PP0.3[4:0]
SUM0.1[6:0]
CARRY0.1[6:0]
SUM0.2[6:0]
CARRY0.2[6:0]
0[3:0]SUM0.1[5:0]
0[3:0]CARRY0.1[5:0]
0[3:0]SUM0.2[6:0]
0[3:0]CARRY0.2[6:0]
PP0.0[4:0]0[3:0]
PP0.1[4:0]0[3:0]
PP0.2[4:0]0[3:0]
PP0.3[4:0]0[3:0]
SUM0.1[5:0]0[3:0]
CARRY0.1[5:0]0[3:0]
SUM0.2[6:0]0[3:0]
CARRY0.2[6:0]0[3:0]

FIG. 14

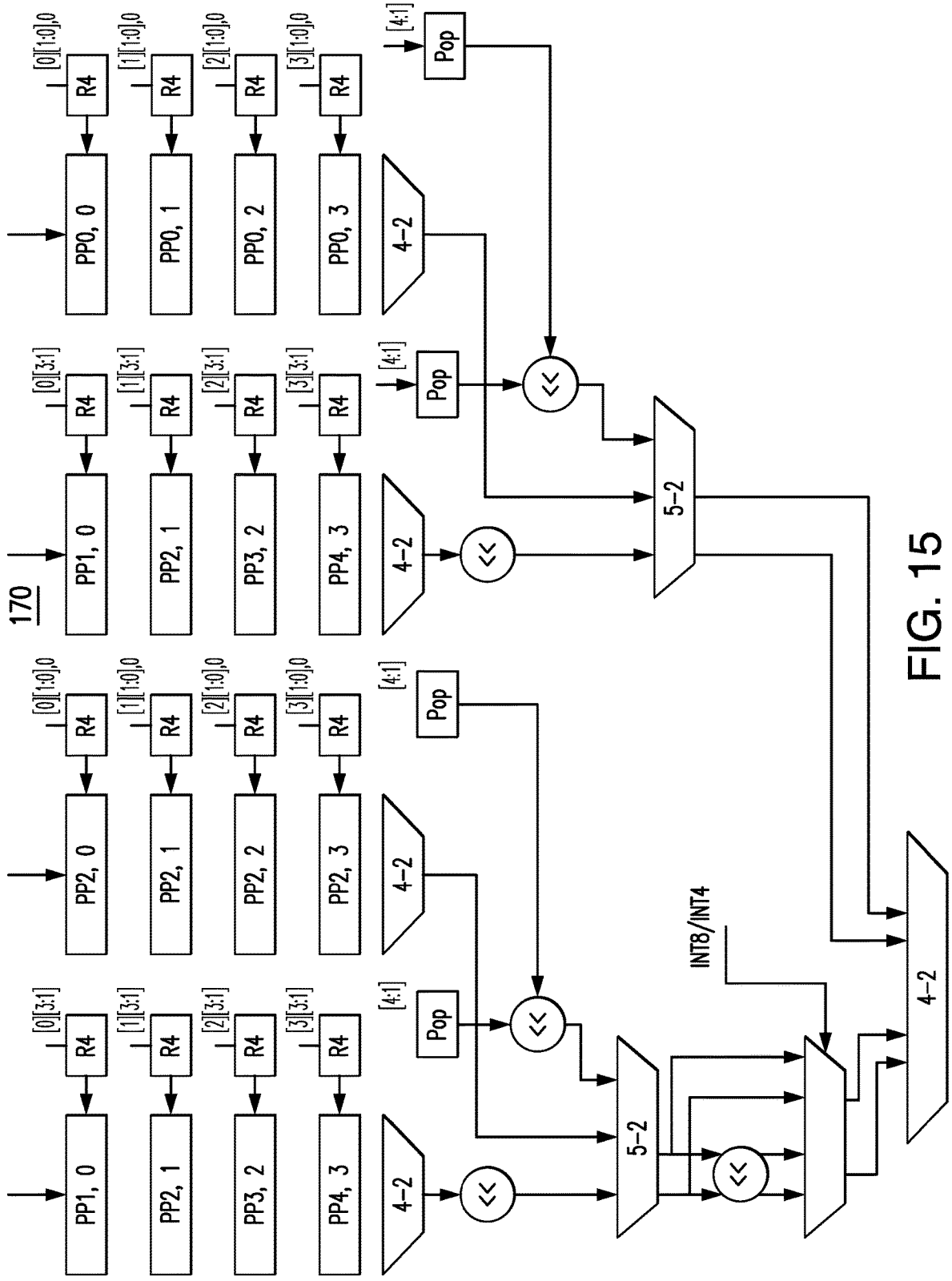

170
PP0, 0
PP0, 1
PP0, 2
PP0, 3
PP1, 0
PP2, 1
PP3, 2
PP4, 3
PP2, 0
PP2, 1
PP2, 2
PP2, 3
PP1, 0
PP2, 1
PP3, 2
PP4, 3
R4
Pop
4–2
5–2
INT8/INT4

FIG. 15

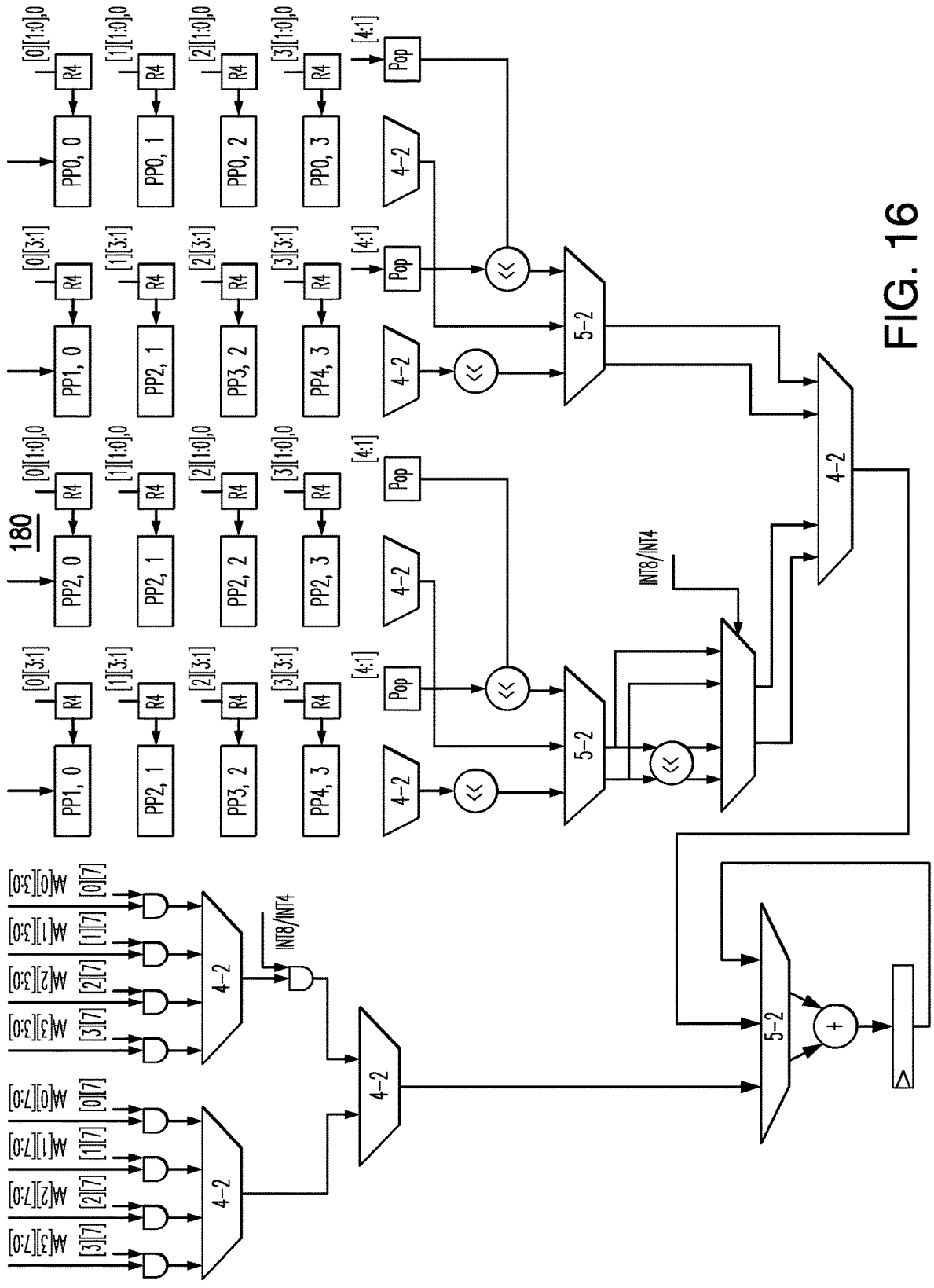

180
PP0, 0
PP0, 1
PP0, 2
PP0, 3
PP1, 0
PP2, 1
PP3, 2
PP4, 3
PP2, 0
PP2, 1
PP2, 2
PP2, 3
R4
Pop
4–2
5–2
INT8/INT4

FIG. 16

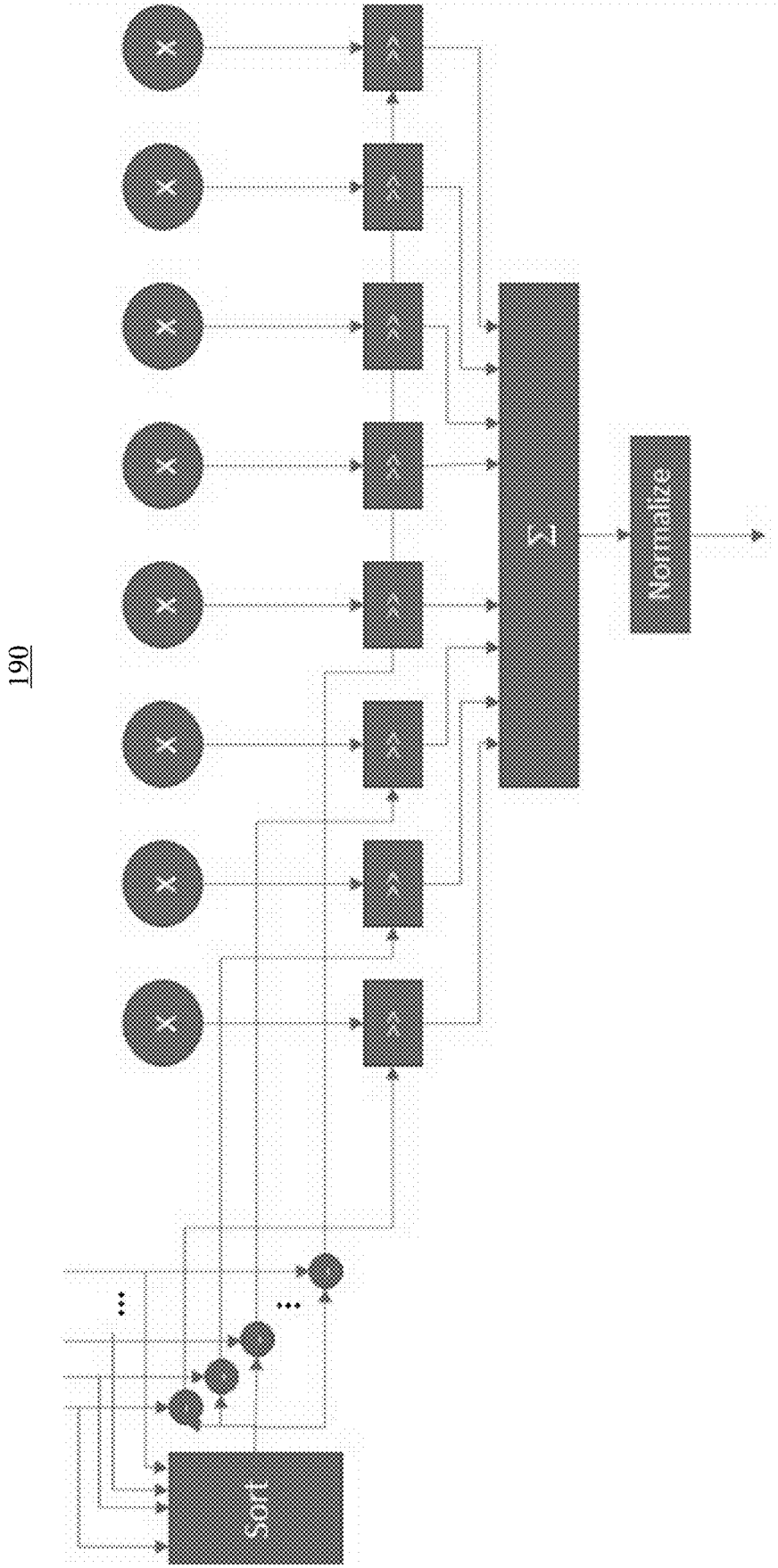


FIG. 17

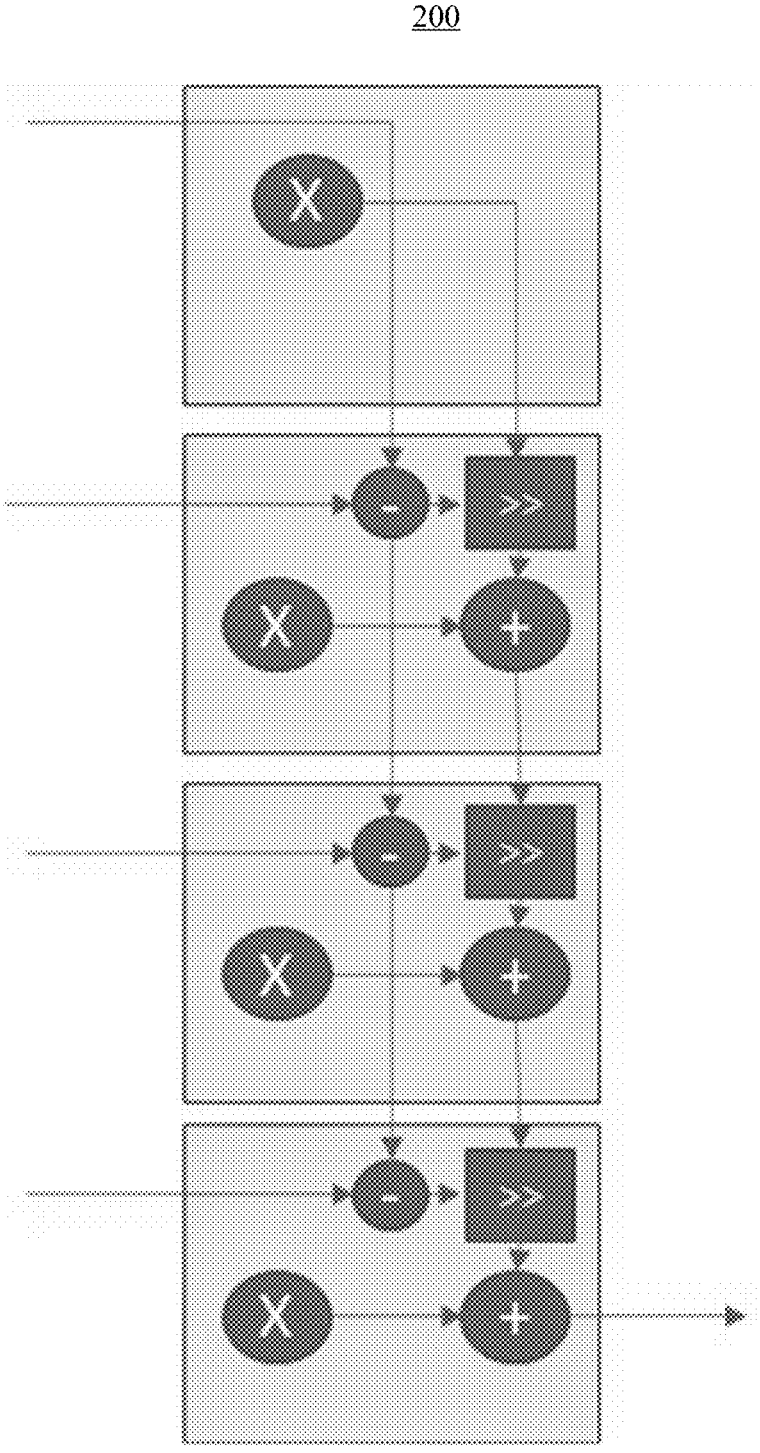


FIG. 18

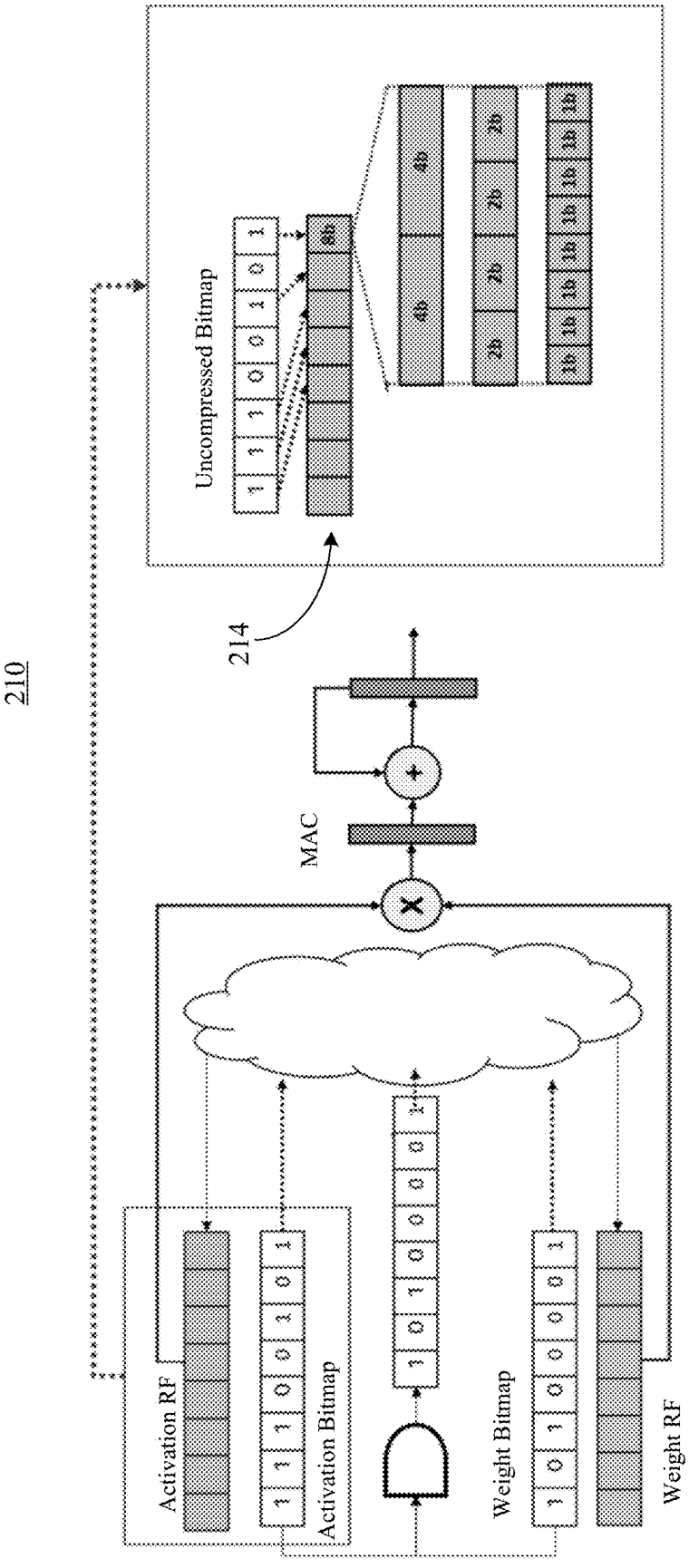

210
214
Uncompressed Bitmap
MAC
Activation RF
Activation Bitmap
Weight Bitmap
Weight RF

FIG. 19

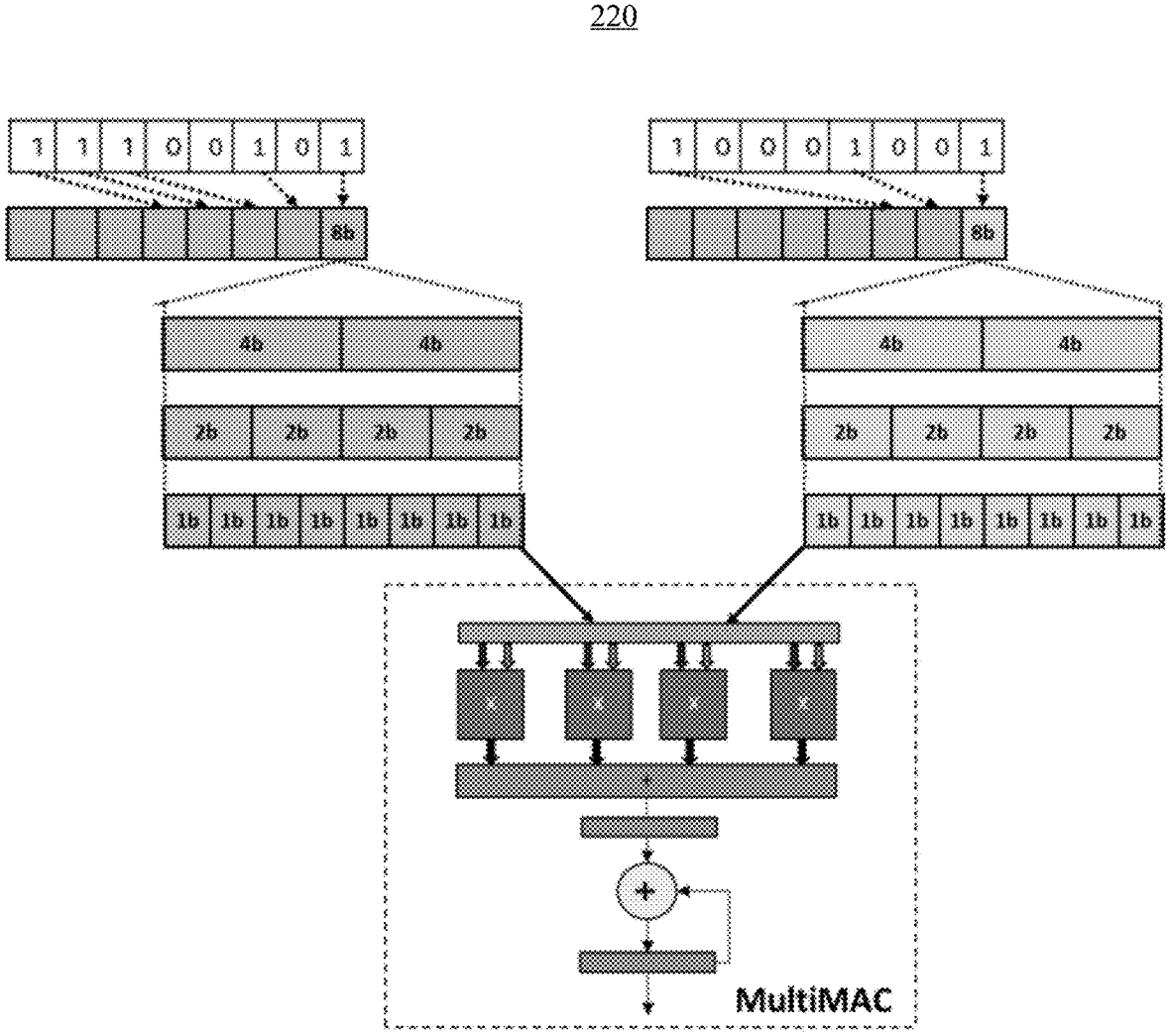

220
1 1 1 0 0 1 0 1
8b
1 0 0 0 1 0 0 1
8b
4b 4b
2b 2b 2b 2b
1b 1b 1b 1b 1b 1b 1b 1b
4b 4b
2b 2b 2b 2b
1b 1b 1b 1b 1b 1b 1b 1b
MultiMAC

FIG. 20

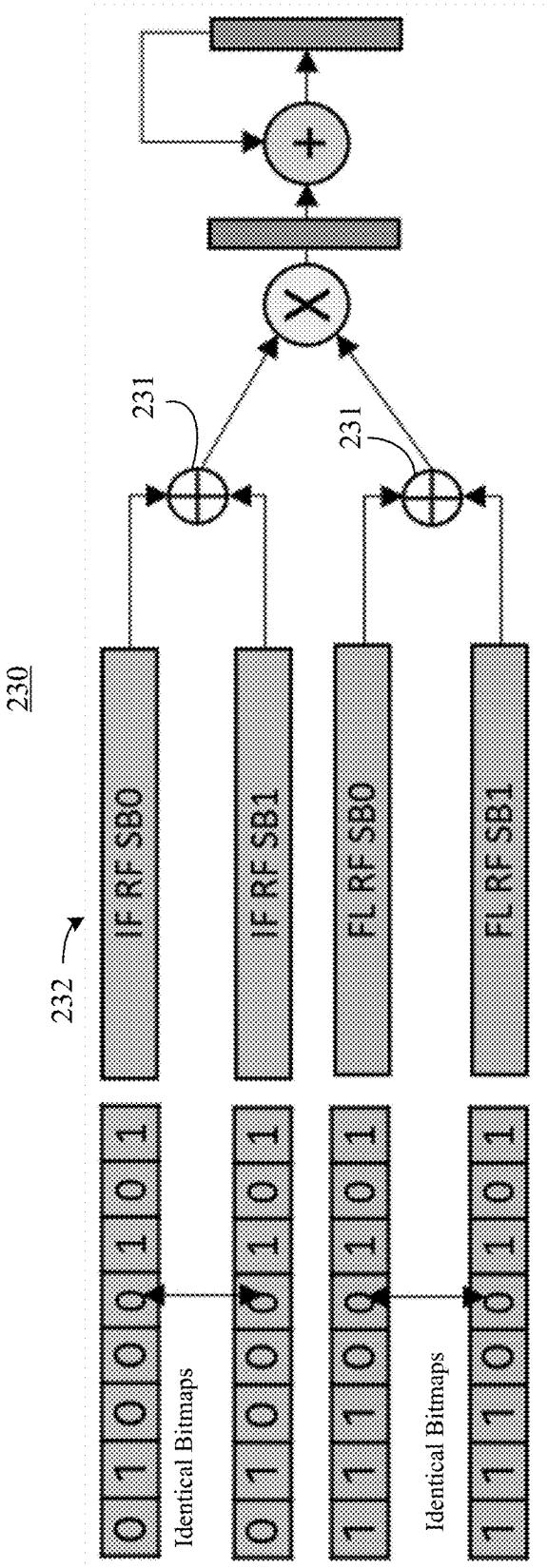

230
232
IF RF SB0
IF RF SB1
FL RF SB0
FL RF SB1
231
231
0 1 0 0 0 1 1 0 1
0 1 0 0 0 1 1 0 1
1 1 1 0 0 1 1 0 1
1 1 1 0 0 1 1 0 1
Identical Bitmaps
Identical Bitmaps

FIG. 21

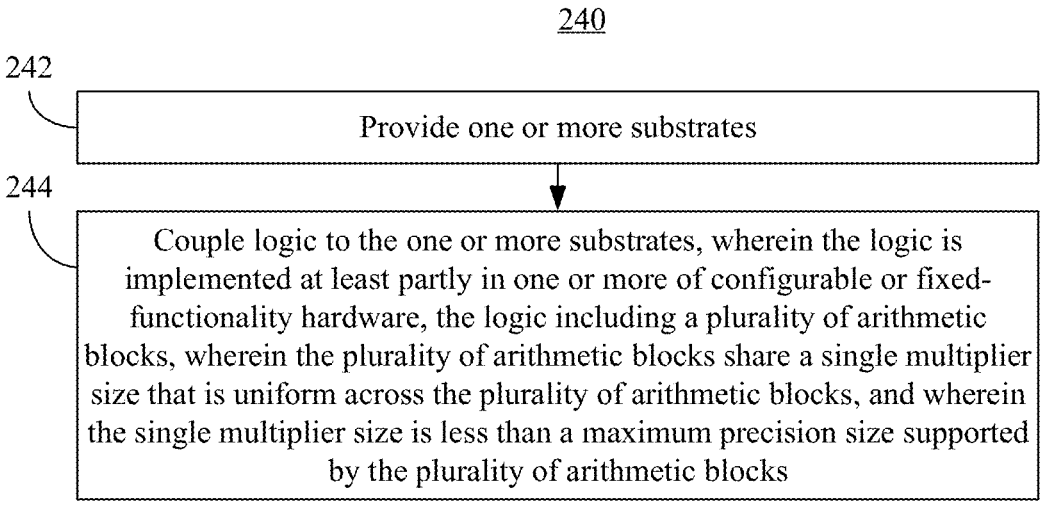


FIG. 22

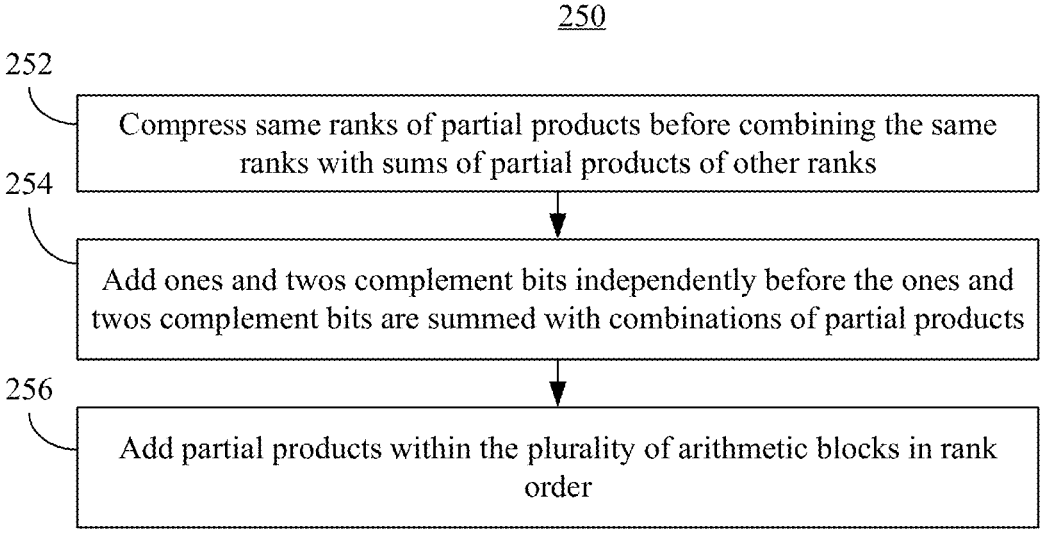


FIG. 23

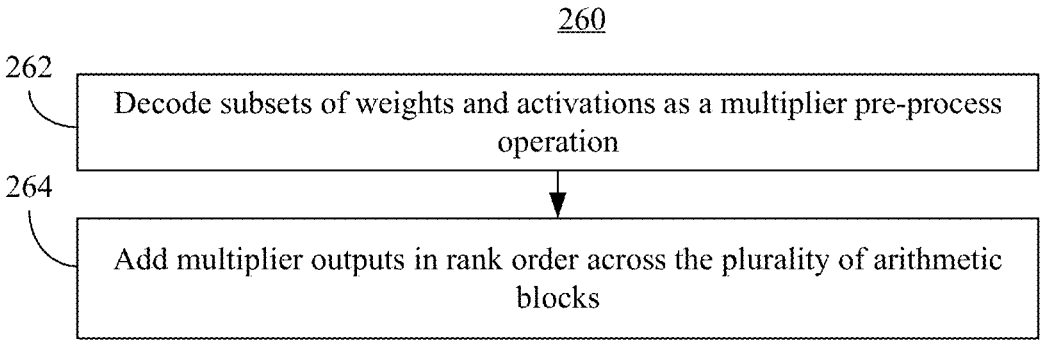


FIG. 24

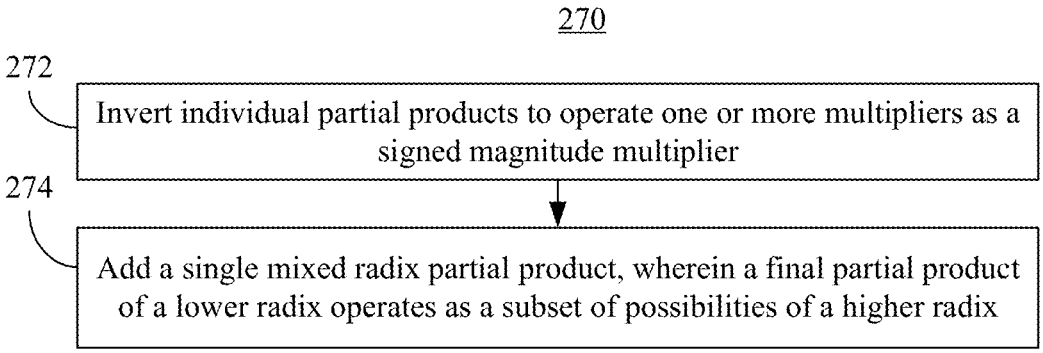


FIG. 25A

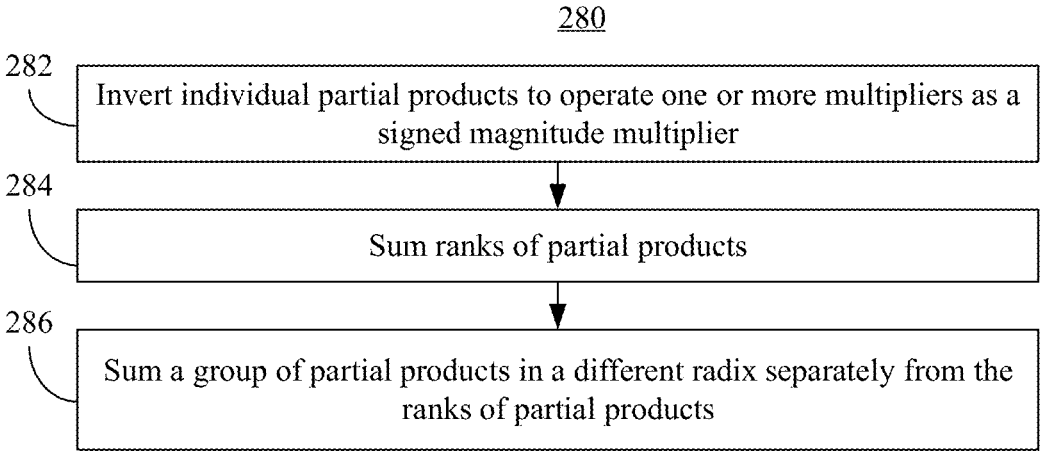


FIG. 25B

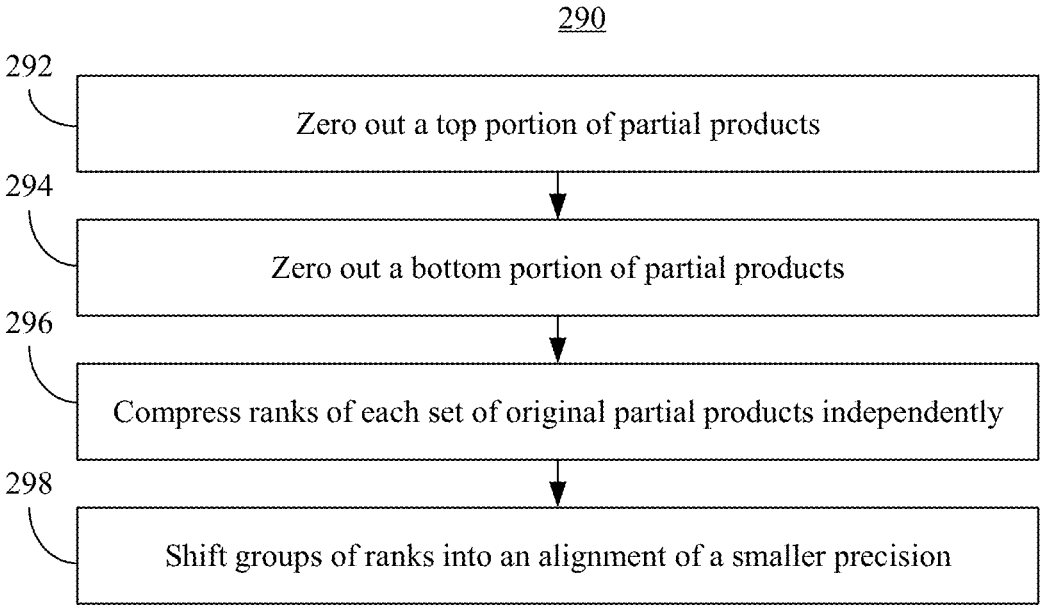


FIG. 26A

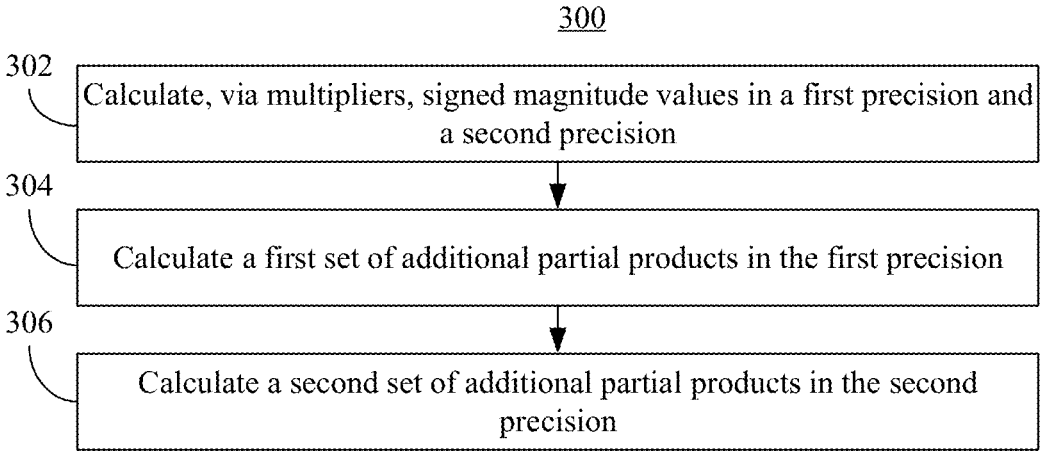


FIG. 26B

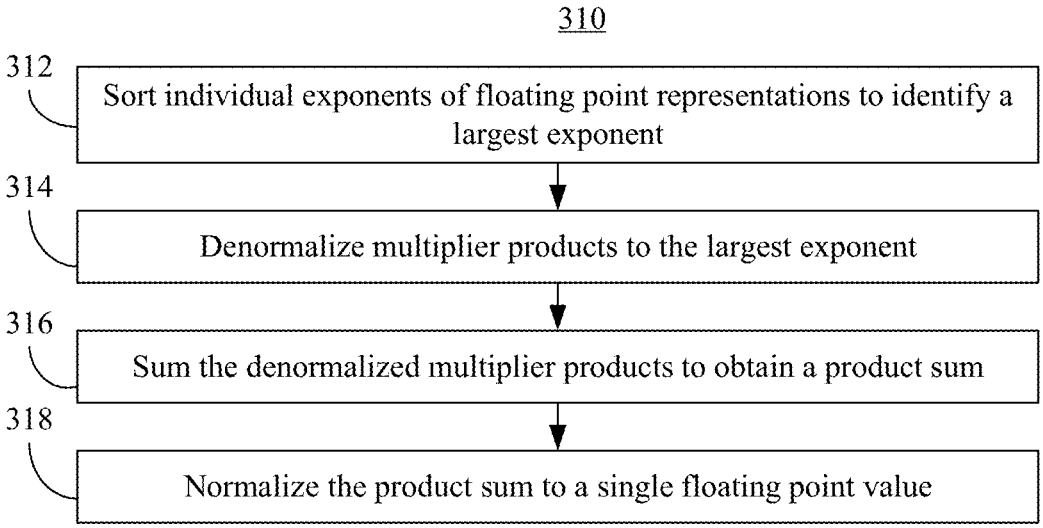


FIG. 27

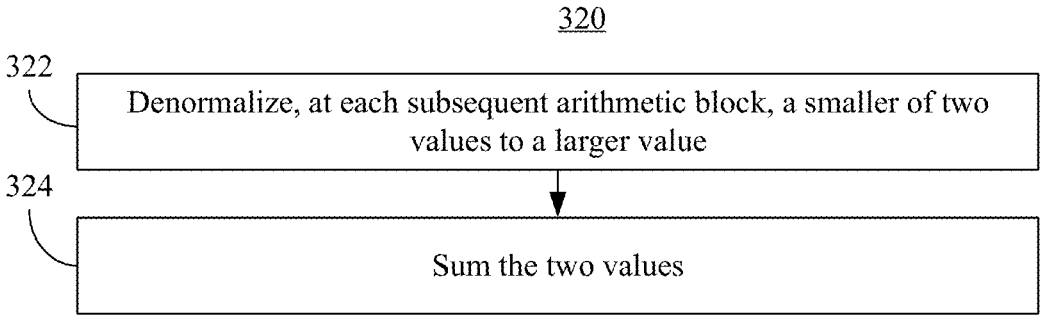


FIG. 28

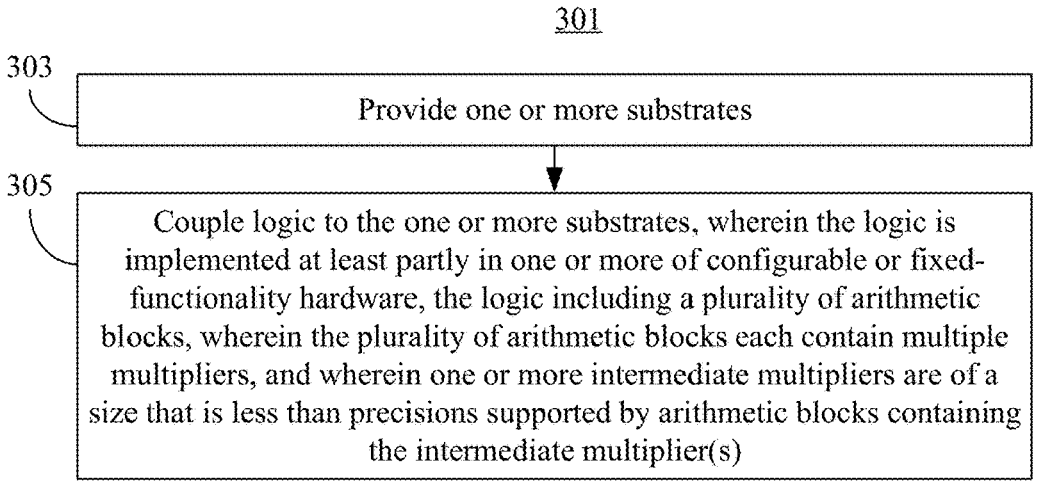


FIG. 29A

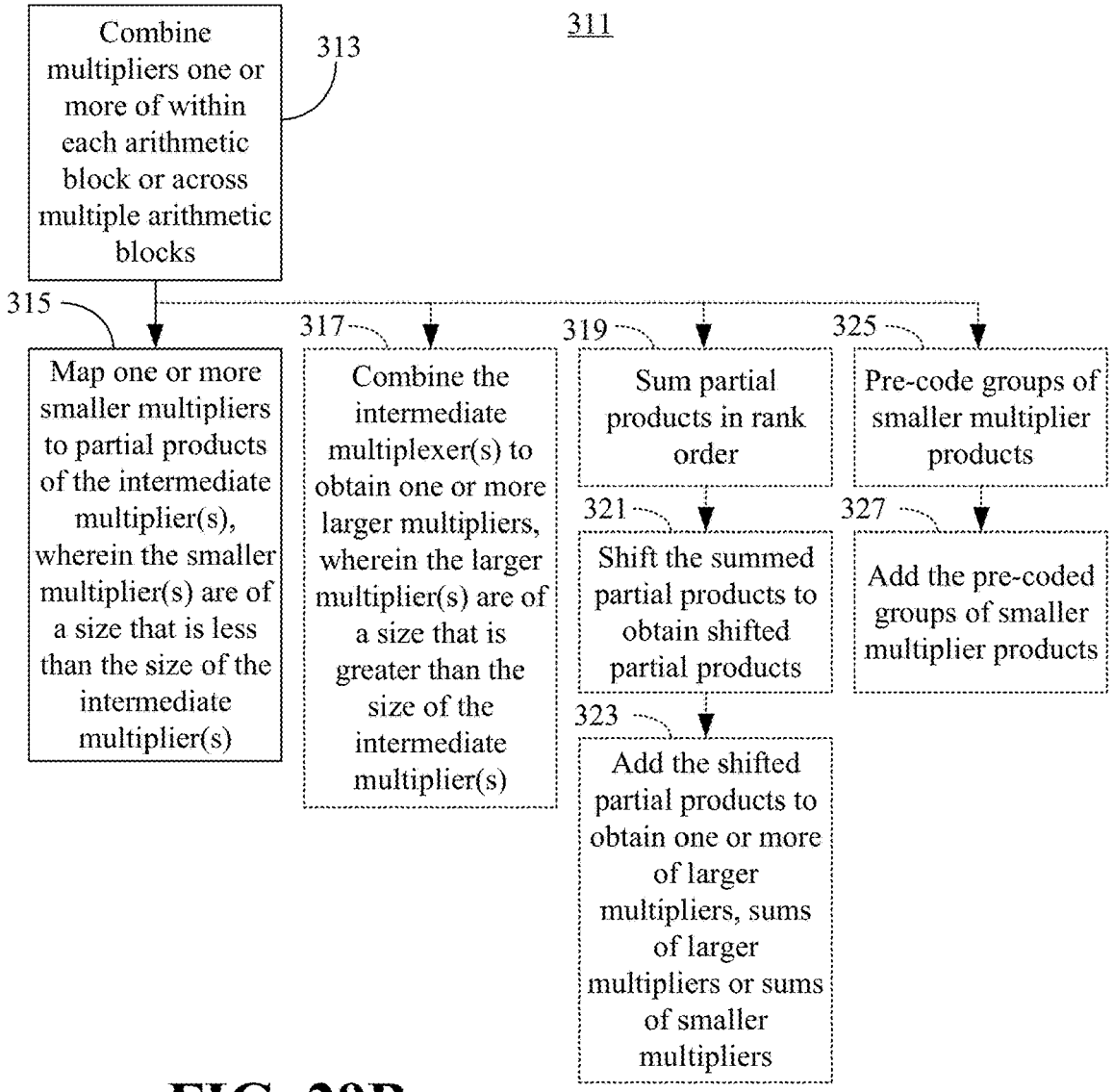


FIG. 29B

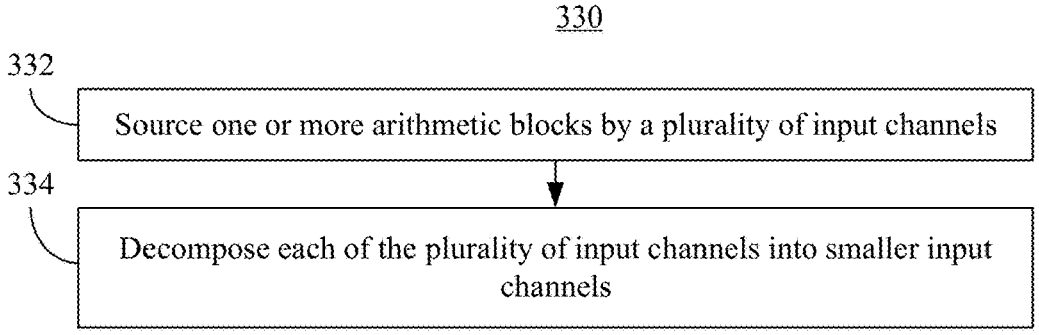


FIG. 29C

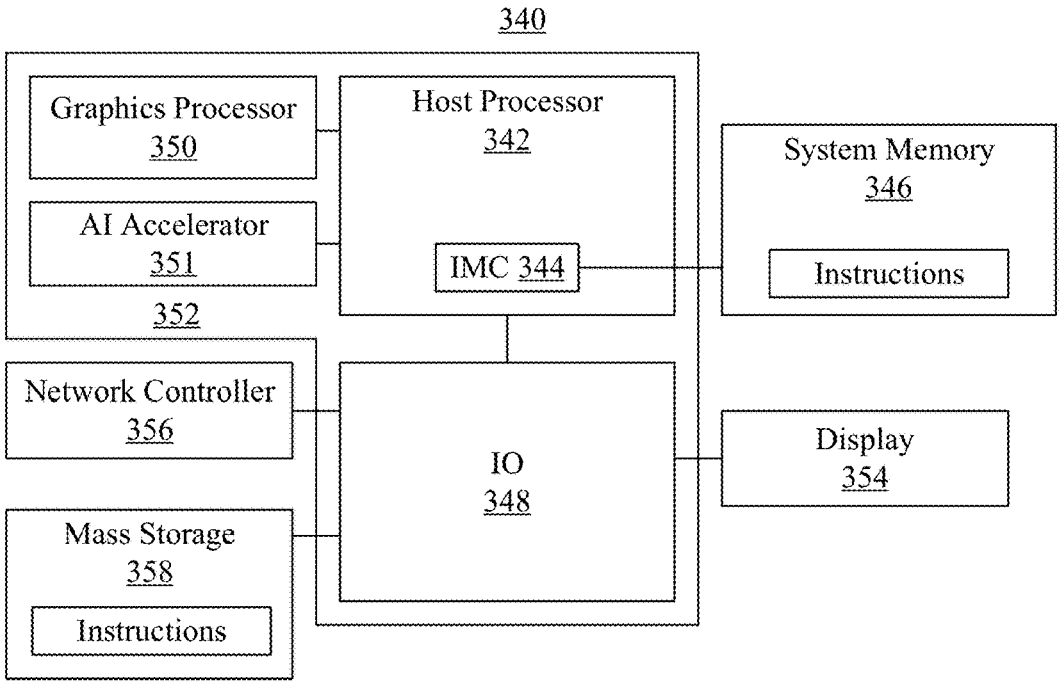


FIG. 30A

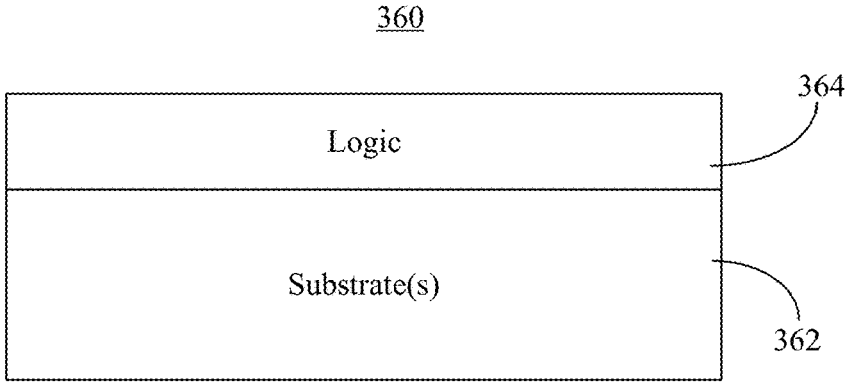


FIG. 30B

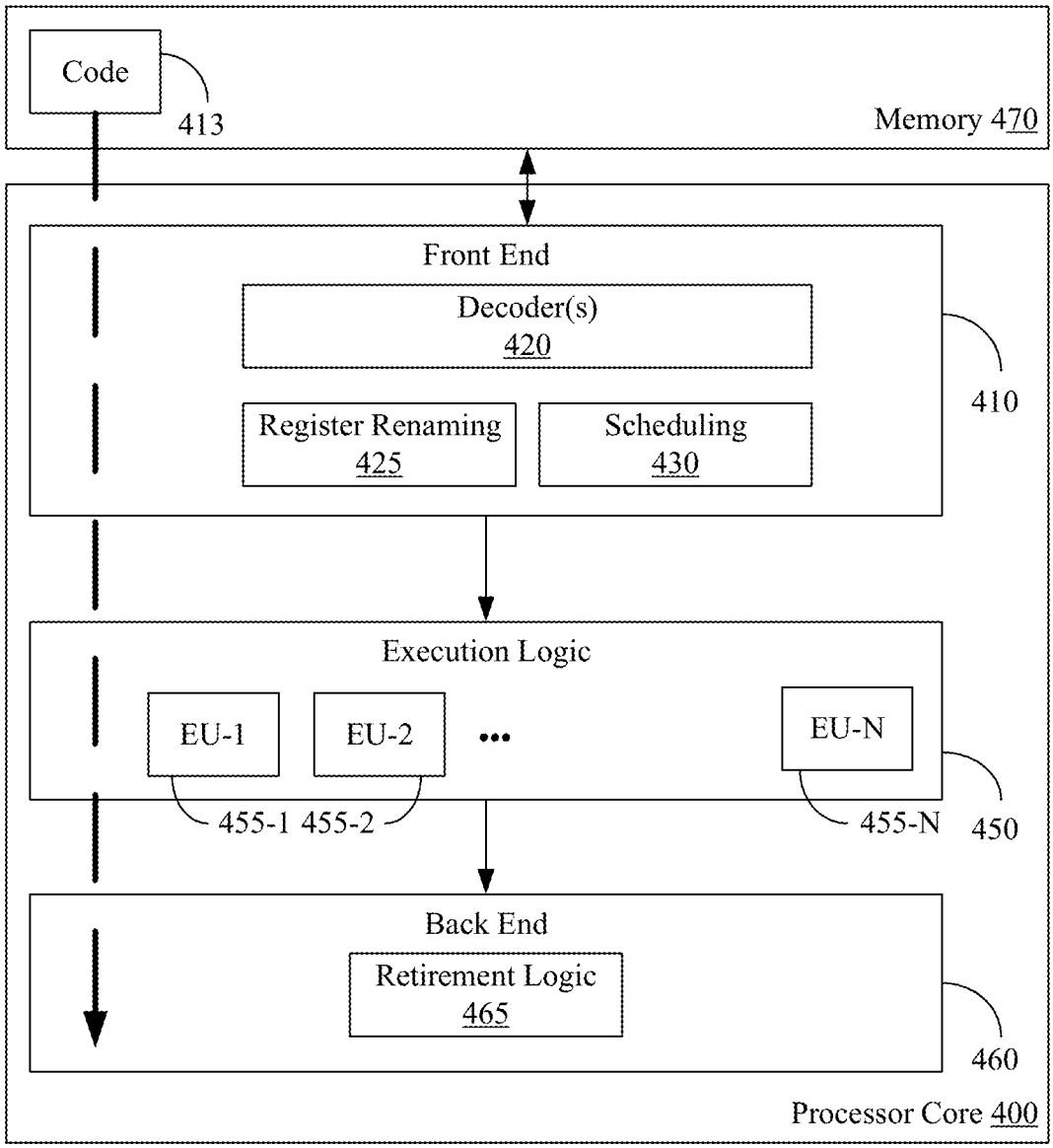
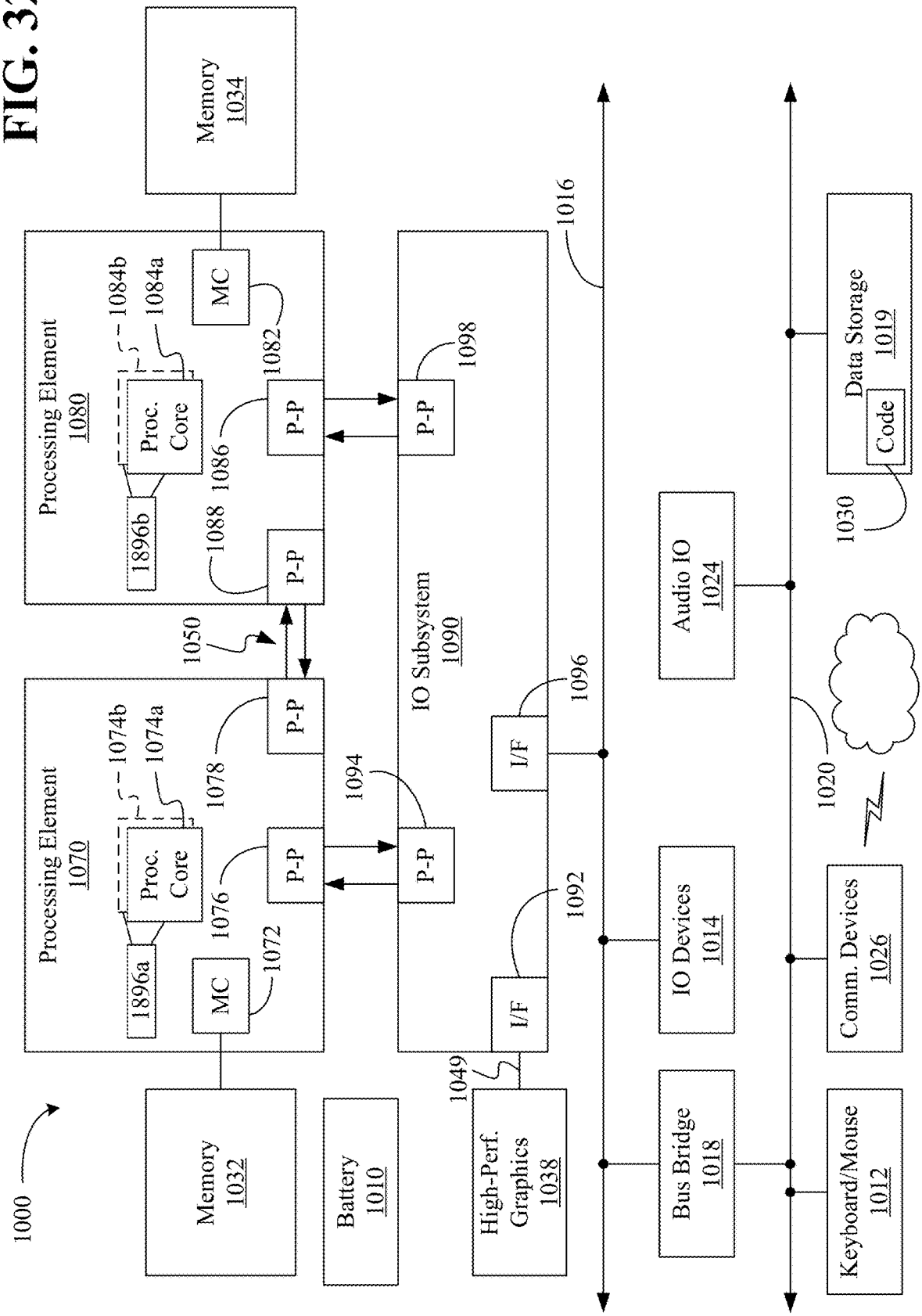

Code
413
Memory 470
Front End
Decoder(s)
420
Register Renaming
425
Scheduling
430
410
Execution Logic
EU-1
EU-2
...
EU-N
455-1 455-2
455-N
450
Back End
Retirement Logic
465
460
Processor Core 400

FIG. 31

FIG. 32


1000
Memory 1032
Battery 1010
Processing Element 1070
Proc. Core
1074a
1074b
18896a
MC
1072
1076
P-P
1078
Processing Element 1080
Proc. Core
1084a
1084b
18896b
MC
1082
1086
1088
P-P
1050
IO Subsystem 1090
P-P
1094
1098
1016
I/F
1096
1092
1049
High-Perf. Graphics 1038
Bus Bridge 1018
IO Devices 1014
Audio IO 1024
Memory 1034
Data Storage 1019
Code
1030
1020
Comm. Devices 1026
Keyboard/Mouse 1012

AREA AND ENERGY EFFICIENT MULTI-PRECISION MULTIPLY-ACCUMULATE UNIT-BASED PROCESSOR

TECHNICAL FIELD

[0001] Embodiments generally relate to multiply-accumulate (MAC) processors. More particularly, embodiments relate to area and energy efficient multi-precision MAC ("MultiMAC") unit-based processors.

BACKGROUND

[0002] Deep Neural Networks (DNN) may be useful for a host of applications in the domains of computer vision, speech recognition, image, and video processing, primarily due to the ability of DNNs to achieve high levels of accuracy relative to human-based computations. The improvements in accuracy, however, may come at the expense of significant computational cost. For example, the underlying deep neural networks typically have extremely high computing demands, as each test input involves on the order of hundreds of millions of MAC operations as well as hundreds of millions of filter weights to be processed for classification or detection.

[0003] As a result, high-end graphics processing units (GPUs) may be suitable to execute these types of workloads because GPUs typically contain thousands of parallel MAC units that can simultaneously operate and produce the output in much less time. GPUs, however, may have very high-power consumption that make them unsuitable to be deployed in highly energy constrained mobile/embedded systems where energy and area budgets are extremely limited.

BRIEF DESCRIPTION OF THE DRAWINGS

[0004] The various advantages of the embodiments will become apparent to one skilled in the art by reading the following specification and appended claims, and by referencing the following drawings, in which:

[0005] FIG. **1** is a block diagram of an example of a MAC processor according to an embodiment;

[0006] FIG. **2** is a comparative block diagram of an example of a conventional single precision MAC unit and a multi-precision MAC unit according to an embodiment;

[0007] FIG. **3** is a comparative block diagram of an example of a conventional multi-precision MAC unit and a multi-precision MAC unit according to an embodiment;

[0008] FIG. **4** is a comparative schematic diagram of an example of a conventional multi-precision MAC unit and a multi-precision MAC unit according to an embodiment;

[0009] FIG. **5**A is a block diagram of an example of a multi-precision MAC unit operating in an 8-bit precision mode according to an embodiment;

[0010] FIG. **5**B is a block diagram of an example of a multi-precision MAC unit operating in a 4-bit precision mode according to an embodiment;

[0011] FIG. **5**C is a block diagram of an example of a multi-precision MAC unit operating in a 2-bit precision mode according to an embodiment;

[0012] FIG. **5**D is a block diagram of an example of a multi-precision MAC unit operating in a binary precision mode according to an embodiment;

[0013] FIG. **6** is a comparative block diagram of an example of a conventional binary data path architecture and a binary data path architecture according to an embodiment;

[0014] FIG. **7** is an illustration of an example of sum values for combinations of activation and weight pairs according to an embodiment;

[0015] FIG. **8** is a comparative block diagram of an example of a conventional binary MAC unit and a multi-precision MAC unit operating in a binary precision mode according to an embodiment;

[0016] FIG. **9** is a block diagram of an example of a multiplier addition within an arithmetic block and across arithmetic blocks according to embodiments;

[0017] FIG. **10** is a block diagram of an example of a Booth encoding radix-4 (R4) multiplier architecture according to an embodiment;

[0018] FIG. **11** is a block diagram of an example of a Booth encoding R4 rank summed multiplier architecture according to an embodiment;

[0019] FIG. **12** is a block diagram of an example of a Booth encoding R4 signed magnitude multiplier architecture according to an embodiment;

[0020] FIG. **13** is a block diagram of an example of a Booth encoding R4 rank order signed magnitude multiplier array architecture according to an embodiment;

[0021] FIG. **14** is an illustration of an example of an integer-4 (INT4) partial product mapping onto integer-8 (INT8) data paths according to an embodiment;

[0022] FIG. **15** is a block diagram of an example of an INT8 and INT4 data path mapping according to an embodiment;

[0023] FIG. **16** is a block diagram of an example of INT8 and INT4 signed magnitude mappings according to an embodiment;

[0024] FIG. **17** is a block diagram of an example of floating point extensions according to an embodiment;

[0025] FIG. **18** is a block diagram of an example of cascaded floating point arithmetic blocks according to an embodiment;

[0026] FIG. **19** is a block diagram of an example of a block sparsity architecture with multi-precision MAC according to an embodiment;

[0027] FIG. **20** is a block diagram of an example of a find-first block sparsity architecture with multi-precision MAC according to an embodiment;

[0028] FIG. **21** is a block diagram of an example of a sparsity architecture working with floating point mode according to an embodiment;

[0029] FIG. **22** is a flowchart of an example of a method of fabricating a MAC processor according to an embodiment;

[0030] FIGS. **23-28** are flowcharts of examples of methods of operating a MAC processor according to an embodiment;

[0031] FIG. **29**A is a flowchart of an example of method of fabricating a MAC processor according to another embodiment;

[0032] FIGS. **29**B-**29**C are flowcharts examples of methods of operating a MAC processor according to other embodiments;

[0033] FIG. **30**A is a block diagram of an example of a performance-enhanced computing system according to an embodiment;

[0034] FIG. **30**B is an illustration of an example of a semiconductor apparatus according to an embodiment;

[0035] FIG. **31** is a block diagram of an example of a processor according to an embodiment; and

[0036] FIG. **32** is a block diagram of an example of a multi-processor based computing system according to an embodiment.

DETAILED DESCRIPTION OF EMBODIMENTS

[0037] The process of quantization can be effective in making relatively large DNN models compact to be deployed on area and energy constrained mobile and other edge devices. Quantization reduces the precision of weights, feature maps, and intermediate gradients from the baseline floating point sixteen/Brain floating sixteen (FP16/BF16) to integer-8/4/2/1 (INT8/4/2/1). Not only does this approach reduce storage requirements by 2×/4×/8×/16×, but the approach also reduces computation complexity by a similar degree that results in a proportional improvement in throughput. As a result, some of the most advanced state-of-the-art DNN accelerators are built with the ability to perform MAC operations of multiple precisions (INT8/4/2/1). The introduction of a multi-precision MAC basically leads to performance improvements that can significantly improve two measurable metrics for DNN accelerators: i) performance per unit area measured using the TOPS/mm^2 (Tera (1012) operations per mm^2) metric, and ii) performance per unit energy measured using the TOPS/W (Tera (1012) operations per Watt) metric.

[0038] Embodiments provide for a DNN processing engine (PE) that can support MAC operations of different precisions, (INT8/4/2/1 to FP16/BF16) while using a common low overhead sparsity acceleration logic. The primary contributions presented by embodiments are related to the ways that data is implemented and fed to the multi-precision MAC unit based on sparsity. Towards that end, "MultiMAC" is an area-efficient, multi-precision multiply and accumulate unit-based processing element in DNN accelerators, where embodiments intelligently share circuit elements among various precision circuits to reduce the area overhead and the energy consumption of the multi-precision MAC unit. The sharing of the circuit elements is enabled by using a data flow that allows input channel-based accumulation (e.g., common in most tensor operations in a DNN accelerator).

[0039] FIG. **1** shows a multi-precision MAC processor **40** that includes a plurality of arithmetic blocks **42** (**42***a*-**42***n*, e.g., arithmetic logic units/ALUs), wherein the plurality of arithmetic blocks **42** share a single multiplier size **44** that is uniform across the plurality of arithmetic blocks **42** (e.g., all multipliers are of the same size). In the illustrated example, each of the arithmetic blocks **42** includes a set of multipliers **46** and each of the multipliers **46** operates on the same number of bits. Additionally, the single multiplier size **44** is less than a maximum precision size supported by the plurality of arithmetic blocks. In one example, the maximum precision size is eight bits (e.g., integer-8/INT8, unsigned integer-8/UINT8) and the single multiplier size **44** is five bits (5b). In an embodiment, the multi-precision MAC processor **40** includes logic (e.g., logic instructions, configurable hardware, fixed-functionality hardware, etc., or any combination thereof) to arrange sparsity information for activations and weights in accordance with a bitmap format that is common to multiple precisions.

[0040] As will be discussed in greater detail, the processor **40** provides an area-efficient, multi-precision multiply and accumulate unit-based processing element for DNN accelerators, where circuit elements are shared among various precision circuits to reduce the area overhead and the energy consumption of the multi-precision MAC processor **40**. For example, only four 5-bit (5b) multipliers **46** are sufficient to support eight different precision modes for MAC operations such as INT8, UINT8, INT4, UINT4, U4_I4, I4_U4, INT2, and INT1. Here, INT1 is effectively binary (BIN) mode with values −1 and 1 represented by 0 and 1. Integration of the processor **40** enables a dense TOPS/mm^2 of an accelerator to be increased by almost 2×, 4×, and 8×, respectively, when executing quantized inferences in INT4/2/1 precision modes.

[0041] Additionally, by recoding activations and weights in groups of 4-bits, a binarized convolution is realized using INT2 hardware or signed −5b multipliers in this case (e.g., by contrast, other approaches typically require separate hardware and data paths to support both binarized and integer convolutions in a single MAC). Embodiments reduce the area of multi-precision MACs that must support both integer and binarized convolutions. Integration of the processor **40** also enables a dense TOPS/W of an accelerator to be increased by 1.87×, 3.75×, and 7.5×, respectively when running quantized inferences in INT4/2/1 precision modes.

[0042] Due to the innovative way of sharing logic for different precisions, the processor **40** may improve the area by 32% at 1.8 GHz compared to existing multi-precision MAC designs. Indeed, the processor **40** also works seamlessly without any additional overhead in coordination with find-first sparsity acceleration techniques via block sparsity. Using this strategy, 1.06×, 1.19×, 1.44× and 1.08×, 1.15×, 1.31× more TOPS/W and TOP S/mm2 improvements are obtained, respectively, over the baseline case (INT8) where the sparsity logic complexity varies proportionally to the number of operands. For 50% sparsity, the proposed accelerator achieves 1.88×, 3.75×, 7.5×, 15× and 1.95×, 3.98×, 7.97×, 15.93× higher TOPS/W and TOPS/mm^2 for INT8/4/2/1 modes compared to accelerators without any MultiMAC support.

[0043] FIG. **2** shows a conventional a single precision (INT8) MAC unit **50** that may form the core of most DNN accelerators. By contrast, an enhanced multi-precision MAC unit **52** supports multiple INT precision modes. In the illustrated example, the conventional MAC unit **50** (e.g., located in each PE of a DNN accelerator) accumulates over input channels (IC). In this mode, the data that is to be accumulated (either belonging to input channels or to a N×N filter window) are fed sequentially one after another from an internal activation register file (RF) **54** and weight register file (RF) **56**.

[0044] Embodiments implement a compute near memory microarchitecture where each PE includes one or more of the enhanced MAC units **52** along with local memory or register files for storing the activations (IF RF) and the filter weights (FL RF). The output activations are stored within the OF RF. In this particular mode of operation, the IF and FL RFs are arranged sequentially in the IC dimension so that the MAC unit within each PE can be fed with these values one after another that are then multiplied and accumulated over time and stored in the OF RF. Due to the existence of this mode of operation, PEs can accumulate over ICs every round which enables the current MultiMAC implementation

where each INT8/byte operand within the RFs can be assumed to be either a single IC or multiple ICs of bitwidth 4, 2, and 1. For IC bitwidths 1, 2, 4, and 8, each byte represents 8, 4, 2, and 1 ICs, respectively. For the sake of simplicity, an IC is consistently split into multiple smaller precision ICs so that it can be accumulated with the help of the MultiMAC. At lower precision modes, the enhanced MAC unit **52** enables the accumulation of multiple ICs (1/2/4/8) in a single clock period. Since the enhanced MAC unit **52** is designed to operate at the same high frequency as that of the single precision MAC unit **50**, the accumulation of multiple ICs in a single clock cycle leads to significantly higher throughput (TOPS). Note that this fixed way of grouping or concatenating lower precision ICs into a single operand involves the least changes or additions in the load, compute, and drain of a typical DNN accelerator to support multi-precision convolutions.

[0045] FIG. **3** shows a conventional multi-precision MAC unit **60** and an enhanced multi-precision MAC unit **62** that maximizes the sharing of circuit logic for supporting INT8/4/2/1 modes. The conventional multi-precision MAC unit **60** implements binary mode using a separate piece of logic **64** that includes mainly bitwise shift and other simple logical operations. In the enhanced multi-precision MAC unit **62**, the logic **64** is completely removed via circuit sharing technology.

[0046] The application of MultiMAC is not limited to the domain of machine learning accelerators. Multi-precision multipliers are also useful for many GPU-specific as well as various image and video processing hardware accelerators. To be more specific, any application that uses multiply-accumulate operations and can tolerate quantization errors such as most applications in the domains of multimedia processing (graphics, video, image, audio, speech, etc.) may benefit from the MultiMAC technology described herein.

[0047] FIG. **4** shows a comparison between a conventional multi-precision MAC unit 8b/4b/2b data path **70** and a MultiMAC enhanced data path **72**. For each 8b weight and activation in the data path **70**, a 9×9b multiplier is used to support signed or unsigned 8b input data. This multiplier is reused for some of the lower precision signed/unsigned 4b/2b multiplication operations ("multiplies"). A separate 5b×5b multiplier is used in 4b and 2b signed/unsigned modes, and two 2b×2b multipliers calculate the remaining portions of the 2b mode multiplies. The four multiplier outputs are summed in an adder tree and then combined with the outputs of the other four 8b channels for a final dot product adder tree. The accumulator is also added at this stage.

[0048] By contrast, the enhanced data path **72** includes multi-precision MAC unit, with no explicit 8b multiplier. Instead, the 8b mode is implemented using four separate 4b quadrant multipliers **71**. Furthermore, rather than complete product computes per channel, sub-products are summed across channels to obtain the same result with lower area. This optimization takes advantage of the fact that intermediate results may be disregarded, with only the final dot-product being relevant. The enhanced data path **72** also ensures minimal reconfiguration multiplexers in the final adder tree. As a result of this dot-product structure, 4b, 2b dot-products are shifted left by 4b. Instead of correcting that shift by reconfiguring the carry-save tree, a non-critical accumulator input is shifted left by 4b and the final result is shift right by 4b in modes lower than 8b. This approach causes adder-tree width to grow by 4b but lower area is obtained overall. Per channel computations are completed before summation across channels in the illustrated enhanced data path **72**.

[0049] FIGS. **5**A-**5**D show the sharing of logic circuits in MultiMAC across different precision modes. More particularly, FIG. **5**A shows a MultiMAC unit **80** operating on eight bits of activation data **81** and eight bits of weight data **83** in INT8/UINT8 modes, FIG. **5**B shows a MultiMAC unit **82** operating in INT4/UINT4 modes, FIG. **5**C shows a MultiMAC unit operating in INT2/UINT2 modes, and FIG. **5**D shows a MultiMAC unit **86** operating in INT1/Binary mode.

[0050] Thus, FIGS. **5**A-**5**D demonstrate the seven modes of MultiMAC operation. The seven modes of operation support seven different datatypes—UINT8, INT8, UINT4, INT4, UINT2, INT2, and INT1 or binary (BIN). Note that independent of the selected datatype, the final output values are consistently accumulated as INT32 (32-bit integers) and are stored in the OF RF. These 32-bit accumulated outputs are finally fed to a PPE to be again converted to the target bitwidth and act as the input activation for the next layer. Note that the MAC units **80**, **82**, **84**, **86** are pipelined internally between the multi-precision multiplier blocks and the accumulator. Moreover, all seven different precision modes use the same set of four signed INT5 multipliers that are used in combination for each of the different precision modes. The illustrated solution can also support several other hybrid precision modes such as UINT4_INT8 and INT8_UINT4 also with the same logic.

[0051] FIG. **6** shows a conventional data path **90** and a conventional convolution architecture **92** in comparison to an enhanced data path **94** and an enhanced convolution architecture **96**. In an embodiment, the binary convolution logic is eliminated by implementing the binary convolution logic via the signed INT2 logic that is already supported within the MultiMAC. Binarized neural networks constrain weights and activations to be +1 or −1. In hardware, +1 is represented as 1b and −1 is represented as 0b. Typical integer (signed/unsigned) convolutions use two's complement arithmetic and hardware to perform the multiply-accumulate (MAC) operation. The integer MAC hardware cannot directly be used to perform the binarized +1/−1 MAC operation. The conventional data path **90** and convolution architecture **92** use dedicated hardware for binary convolution without sharing any logic with higher precision MAC hardware.

[0052] The conventional convolution architecture **92** shows an example circuit to perform the MAC operation for a byte of activations and a byte of weights, where each byte contains eight binary activations/weights. First, the activations and weights are XOR-ed together and the number of ones in the result is counted. The number of ones counted in the XOR-ed result is used to generate an index into a LUT (lookup table) and the LUT returns the sum of the binarized products in the range {−8, −6, −4, 2, 0, 2, 4, 6, 8}. The LUT output is added to the accumulator to produce the MAC result. This approach requires a separate binary datapath and a separate integer datapath in the MAC.

[0053] With continuing reference to FIGS. **6** and **7**, however, a sum values table **98** provides some hints to for the binary convolution to share logic with INT2 convolution. More particularly, the table **98** considers two bits of activations and two bits of weights. The final SUM=PROD[1]+PROD[0], where PROD[0]=ACT[0]*WT[0] and PROD[1]

=ACT[1]*WT[1]. The intermediate MULT is created as a decode of {ACT[1:0], WT[1:0]} using a lookup table (LUT) **100** and a constant (CONST) **102**, which is fixed at −2. The final RESULT=MULT*CONST and gives same result as SUM. As result, the dedicated binary data path is no longer needed in MAC units and can be entirely eliminated. In an embodiment, the dedicated binary data path is replaced by a 16-item look up operation to drive inputs into INT2 MAC hardware. This approach scales to n-bit but the size of the lookup increases exponentially (e.g., INT4 involves a 256-item lookup). There are both area and energy benefits of this approach. Thus, FIGS. **6** and **7** demonstrate how the binary logic may be implemented using INT2 logic and 5b multipliers. FIG. **8** shows a side-by-side comparison of the conventional binary logic **64** and the MultiMAC unit **86** operating in INT1/Binary mode with a common 5-bit multiplier.

[0054] FIG. **9** demonstrates that embodiments may support higher precision operations by sharing circuitry from lower precision ones in a more generic way. More particularly, the technology described herein may be applied to a single functional unit **110** (arithmetic block, ALU block), or across multiple functional units **112**. Sums of multipliers can be implemented locally, or across many normally independent paths. More efficient ways of implementing the sums may be obtained by grouping multi-operand additions by rank (e.g., bit position) subsets first.

[0055] FIG. **10** shows a structure of an application specific integrated circuit (ASIC) multiplier architecture **120** that is implemented based on the Booth encoding radix-4 (R4) multiplier. The multiplicand is used to create partial products 122 ("PP0"-"PP3"), based on tri-bit encodings of the multiplier operand. The partial products are added. In one example, the partial products are compressed in redundant form using a 4-2 compressor **124** or a 3-2 compressor **126**, usually in a Wallace tree or Dadda tree, although other compressors, counters, and tree topologies may be used. The two final redundant vectors are added by a carry propagate adder **128**.

[0056] FIG. **11** shows an architecture **130** in which the outputs of several multipliers are added together efficiently. Rather than treating each multiplier separately, all of the same ranks of partial products are compressed first (e.g., initial 4-2 compression). In the illustrated example, "PP4,0" refers to partial product four, rank zero, "PP4, 1" refers to partial product four, rank one, and so forth. The ranks are summed (e.g., typically in redundant format, but the rank sums could also be added by a carry-propagate adder/CPA at the bottom of the compression of each rank). In many cases, the negative partial products will have an associated "+1" which will turn the negative partial products from 1's complement to 2's complement values. The 2's complement values are usually compressed as part of the tree. Instead, the architecture **130** performs a population ("pop") count for all of the +1s for any given rank and adds the result into the sums at the bottom. The pop counts can be added or compressed into each rank result separately, or all pop counts can be added or otherwise combined together and then added or compressed into the entire structure at a single point.

[0057] Thus, the architecture **130** and/or the architecture **120** (FIG. **10**) may compress same ranks of partial products before combining the same ranks with sums of partial products of other ranks and adds ones and twos complement bits independently before the ones and twos complement bits are summed with combinations of partial products. In an embodiment, the architecture **130** also adds partial products within a plurality of arithmetic blocks in rank order.

[0058] FIG. **12** shows the extension of the multiplier of the architecture **130** (FIG. **11**) to an architecture **140** that performs signed magnitude multiplication, which can be extremely useful for artificial intelligence (AI) applications. In the illustrated example, a separate sign bit is used to indicate the sign of an unsigned number, such as used in IEEE 754 floating point (754-2019—IEEE Standard for Floating-Point Arithmetic, 22 Jul. 2019). There are two aspects of this implementation.

[0059] If the product is negative (e.g., one but not both inputs are negative), the Booth encoding for every partial product is inverted (along with the 1's to 2's bit). In addition, the multiplicand is added to tree reduction, and shifted two bits to the left of the highest Booth R4 coded partial product, but only if the most significant bit of the multiplier operand is "1" (e.g., the operand value is negative). This approach is taken because an unsigned multiplier (which is negated or not by the signed magnitude signs) is used via a mixed radix extension. All of the partial products of the multiplier are in Booth R4 format, but the uppermost partial products is in radix-2 (R2) format. Another way of looking at this condition is that the uppermost partial product is coded as a subset of Booth R4, where only the least significant bit (lsb) of the uppermost tri-bit (which is the most significant bit/msb of the penultimate tri-bit) is considered. A different way of explaining this approach is that if the msb of the multiplicand is "1", then the next Booth tribit would be "001", or (+1× multiplicand) and if the msb was "0" then the tribit would be "000" (0x multiplicand).

[0060] FIG. **13** shows an architecture **150** that is extended to a rank ordered multiplier array. In the illustrated example, the final partial products are all summed separately from the rest of the arrays. Embodiments support arrays of smaller multipliers, which can be extracted from the larger multipliers array. For example, two INT4 multipliers may be extracted from one INT8 multiplier. The arithmetic complexity of an INT8 multiplier is actually 4× that of a INT4 multiplier, but to access the additional two INT4 multipliers would involve 2× the number of input wires.

[0061] Thus, the architecture **150** and/or the architecture **140** (FIG. **12**) may invert individual partial products to operate one or more multipliers as a signed magnitude multiplier. Additionally, the architecture **150** and/or the architecture **140** (FIG. **12**) may add a single mixed radix partial product, wherein a final partial product of a lower radix operates of a subset of possibilities of a higher radix.

[0062] In addition, the architecture **150** may, for a group of multipliers, invert individual partial products to operate one or more multipliers as a signed magnitude multiplier and sum ranks of partial products. In an embodiment, the architecture **150** sums a group of partial products in a different radix separately from the ranks of partial products. In an embodiment, the group of multipliers provide unsigned multiplication. The group of multipliers may also be in an unsigned magnitude format.

[0063] FIG. **14** demonstrates that to save power, a mapping **160** may leave the respective lsbs and msbs at zero when the INT4 operands are mapped to the INT8 partial products. Thus, the area of natural word growth is extracted from the redundant compression.

[0064] FIG. **15** shows the mapping and alignment selection for the INT8 and INT4 data paths in an architecture **170**. In an embodiment, the architecture **170** zeroes out a top portion of partial products, zeroes out a bottom portion of partial products, and compresses ranks of each set of original partial products independently. The architecture **170** may also shift groups of ranks into an alignment of a smaller precision.

[0065] FIG. **16** shows an architecture **180** of a signed magnitude implementation for the INT4 values as a follow on from the INT8 signed magnitude calculations and the INT8/INT4 mappings. The same signed magnitude R2 extensions are used for one half of the INT4 multipliers. A second set of partial product extensions are provided for the other half of the INT4 multipliers. In the illustrated example, the partial product extensions are only 4b wide. In INT4 mode, other half will be of the same rank as the upper half of the INT8 extensions, and will be added there.

[0066] In an embodiment, the architecture **180** calculates, via multipliers, signed magnitude values in a first precision and a second precision. Additionally, the architecture **180** may calculate/determine a first set of additional partial products in the first precision and calculate/determine a second set of additional partial products in the second precision.

[0067] FIG. **17** shows an architecture **190** in which multiple fixed point multiplies are converted into a floating point dot product relatively inexpensively. Rather than providing a separate floating point adder for each multiplier, all multiply products are denormalized with respect to the largest product. The largest product is found by sorting all of the output exponents of each multiplier. Each exponent is then subtracted from the largest exponent, and each product is then right shifted by the difference. If the difference is larger than the output mantissa, that product can be zeroed, or alternately, used to set a "sticky" bit for that value. All of the values can then be summed together—this summation can be done efficiently by compressing in a redundant form, with a single CPA at the end. Various rounding, error handling, and exception handling functions can be added, but the basic implementation is unchanged. The illustrated architecture **190** may be used both inside a functional block, or across multiple functional blocks.

[0068] In an embodiment, the architecture **190** sorts individual exponents of floating point representations to identify a largest exponent. Additionally, the architecture **190** denormalizes multipier products to the largest exponent and sums the denormalized multiplier products to obtain a product sum. In one example, the architecture **190** normalizes the product sum to a single floating point value.

[0069] FIG. **18** demonstrates that an alternative to the architecture **190** (FIG. **20**) is to extend the solution across multiple blocks, with a complete FP adder being implemented at every block as shown in an architecture **200**. Here, arithmetic blocks are cascaded together. Each block takes the exponent and product of the preceding block and compares the input to an exponent of the block in question. The smaller product (e.g., mantissa) is denormalized and added to the larger mantissa. This value, along with the larger exponent, is forwarded to the next block. In an embodiment, no normalization is applied until the final block in the cascaded chain.

[0070] Sometimes, this approach will not be accurate enough. Thus, the sum at each block can be normalized (which is a larger circuit than not normalizing), but the exception handling and rounding may be bypassed (e.g., only the information is forwarded). This approach may reduce the cost of each floating point adder by 10%-20% over a typical solution (until the final block, which uses a full FP adder).

[0071] In an embodiment, the architecture **200** denormalizes, at each subsequent arithmetic block, a smaller of two values to a larger value. Additionally, the architecture **200** may sum the two values.

[0072] FIG. **19** shows an architecture **210** in which MultiMAC works seamlessly with the sparsity acceleration logic within a PE. In an embodiment, MultiMAC works without any additional changes because both the inner sparsity logic and the MultiMAC unit work on the assumption that IF and FL RFs store ICs sequentially, which will be accumulated within the PE.

[0073] Sparsity logic **212** (e.g., find-first sparsity logic) works with compressed data (e.g., zero-value compressed). The zero and non-zero positions in the activation and weight data are represented by a bit in the bitmap in a compressed mode. The non-zero values are compressed and kept adjacent to one another in an IF RF **214**. In the single precision MAC, each byte represents one activation or filter point and is represented by one bit in the bitmap. The same logic can be kept intact and easily be applied for MultiMAC by introducing the concept of block sparsity where each bit in bitmap can either represent 1, 2, 4, or 8 ICs based on whether UINT8/INT8, UINT4/INT4, UINT2/INT2, or binary mode (BIN), respectively, are active. Only in the case when all ICs or the entire byte is 0, will a 0 be placed in the bitmap (e.g., otherwise the value will be a 1). This coarse-granular approach to maintaining sparsity information for lower precision modes may have pros and cons. For example, one advantage is that the same sparsity encoder that operates at a byte-level may be used, which decreases the overall impact on DNN accelerator area and energy. Another advantage is that the storage and processing overhead of the bitmap for each IC is also reduced at lower precisions. A downside of block sparsity, however, may be that it keeps track of sparsity at a much coarser-granularity and therefore reduces the maximum potential speedup that can be achieved through fine-granular tracking.

[0074] FIG. **20** shows an architecture **220** in which the block sparsity accumulation works within the MultiMAC PE.

Block Sparsity Support for Existing Floating Point Operation

[0075] In addition to the integer-based MultiMAC, support may be provided for floating point execution within the PE. Although this support may involve a completely separate floating point MAC (FPMAC, e.g., separate from the MultiMAC, is not shared), the existing sparsity logic may be readily used for floating point execution.

[0076] FIG. **21** shows an architecture **230** in which floating point (FP16/BF16) execution occurs within the PE. Since each RF subbank (SB) **232** has sixteen 1-byte entries and each bitmap sublane has a bit corresponding to each byte in the RF subbank, a single FP16/BF16 operand may be created by concatenating **231** 1B each from two RF subbanks as shown. In an embodiment, the sparsity logic works "out of the box" without any additional changes. The architecture **230** merely ensures that during zero value

suppression, the higher and lower bytes of a single BF/FP16 operand are not independently encoded. In one example, a zero is only assigned to to a byte when both the upper and the lower halves of the operand are zero (e.g., when the entire activation is zero). Such an approach, ensures that the bitmap fed in the two bitmap sublanes corresponding to the upper and lower bytes of the FP operand are exactly the same. The reuse of sparsity logic for the FP case reduces the overall overhead of sparsity.

[0077] FIG. **22** shows a method **240** of fabricating a performance-enhanced MAC processor. The method **240** may generally be used to fabricate a multi-precision MAC processor such as, for example, the MAC processor **40** (FIG. **1**), already discussed. More particularly, the method **240** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as random access memory (RAM), read only memory (ROM), programmable ROM (PROM), firmware, flash memory, etc., in configurable logic such as, for example, programmable logic arrays (PLAs), field programmable gate arrays (FPGAs), complex programmable logic devices (CPLDs), in fixed-functionality logic hardware using circuit technology such as, for example, application specific integrated circuit (ASIC), complementary metal oxide semiconductor (CMOS) or transistor-transistor logic (TTL) technology, or any combination thereof. The method **240** may also be implemented via suitable semiconductor processes such as, for example, deposition, cutting and/or etching techniques.

[0078] For example, computer program code to carry out operations shown in the method **240** may be written in any combination of one or more programming languages, including an object oriented programming language such as JAVA, SMALLTALK, C++ or the like and conventional procedural programming languages, such as the "C" programming language or similar programming languages. Additionally, logic instructions might include assembler instructions, instruction set architecture (ISA) instructions, machine instructions, machine dependent instructions, microcode, state-setting data, configuration data for integrated circuitry, state information that personalizes electronic circuitry and/or other structural components that are native to hardware (e.g., host processor, central processing unit/CPU, microcontroller, etc.).

[0079] Illustrated processing block **242** provides one or more substrates such as, for example, silicon, sapphire, gallium arsenide, etc. Processing block **244** couples logic (e.g., transistor array and other integrated circuit/IC components) to the substrate(s). In the illustrated example, the logic is implemented at least partly in one or more of configurable or fixed-functionality hardware. Moreover, the logic includes a plurality of arithmetic blocks (e.g., ALUs), wherein the plurality of arithmetic blocks share a single multiplier sized that is uniform across the plurality of arithmetic blocks. Additionally, the single multiplier size is less than the maximum precision size supported by the plurality of arithmetic blocks. In an embodiment, the maximum precision size is eight bits and the single multiplier size is five bits. In one example, block **244** includes arranging sparsity information for activations and weights in accordance with a bitmap format that is common to multiple precisions. The method **240** therefore enhances performance at least to the extent that single multiplier size renders the MAC processor more area and/or energy efficient.

[0080] FIG. **23** shows another method **250** of operating a performance-enhanced MAC processor such as, for example, the multi-precision MAC processor **40** (FIG. **1**), already discussed. The method **250** may generally be implemented in a logic architecture such as, for example, the architecture **120** (FIG. **10**) and/or the architecture **130** (FIG. **11**), already discussed. More particularly, the method **250** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality logic hardware using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof.

[0081] Illustrated processing block **252** compresses same ranks of partial products before combining the same ranks with sums of partial products of other ranks, wherein block **254** adds ones and twos complement bits independently before the ones and twos complement bits are summed with combinations of partial products. In an embodiment, block **256** adds partial products within a plurality of arithmetic blocks in rank order.

[0082] FIG. **24** shows another method **260** of operating a performance-enhanced MAC processor such as, for example, the multi-precision MAC processor **40** (FIG. **1**), already discussed. The method **260** may generally be implemented in a data path such as, for example, the enhanced data path **94** (FIG. **6**) and/or the MultiMAC unit **86** (FIG. **8**), already discussed. More particularly, the method **260** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality logic hardware using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof.

[0083] Illustrated processing block **262** decodes subsets of weights and activations as a multiplier pre-process operation. Additionally, block **264** adds multiplier outputs in rank order across a plurality of arithmetic blocks.

[0084] FIG. **25**A shows another method **270** of operating a performance-enhanced MAC processor such as, for example, the multi-precision MAC processor **40** (FIG. **1**), already discussed. The method **270** may generally be implemented in a logic architecture such as, for example, the architecture **140** (FIG. **12**) and/or the architecture **150** (FIG. **13**), already discussed. More particularly, the method **270** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality logic hardware using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof.

[0085] Illustrated processing block **272** inverts individual partial products to operate one or more multipliers as a signed magnitude multiplier. Additionally, block **274** may add a single mixed radix partial product, wherein a final partial product of a lower radix operates of a subset of possibilities of a higher radix.

[0086] FIG. **25**B shows another method **280** of operating a performance-enhanced MAC processor such as, for example, the multi-precision MAC processor **40** (FIG. **1**), already discussed. The method **280** may generally be imple-

mented, for a group of multipliers, in a logic architecture such as, for example, the architecture **150** (FIG. **13**), already discussed. More particularly, the method **280** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality logic hardware using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof.

[0087] Illustrated processing block **282** inverts individual partial products to operate one or more multipliers as a signed magnitude multiplier, wherein block **284** sums ranks of partial products. Additionally, block **286** sums a group of partial products in a different radix separately from the ranks of partial products. In an embodiment, the group of multipliers provide unsigned multiplication. The group of multipliers may also be in an unsigned magnitude format.

[0088] FIG. **26**A shows another method **290** of operating a performance-enhanced MAC processor such as, for example, the multi-precision MAC processor **40** (FIG. **1**), already discussed. The method **290** may generally be implemented in a logic architecture such as, for example, the architecture **170** (FIG. **15**), already discussed. More particularly, the method **290** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality logic hardware using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof.

[0089] Illustrated processing block **292** zeroes out a top portion of partial products, wherein block **294** zeroes out a bottom portion of partial products. In one example, block **296** compresses ranks of each set of original partial products independently. Block **298** may shift groups of ranks into an alignment of a smaller precision.

[0090] FIG. **26**B shows another method **300** of operating a performance-enhanced MAC processor such as, for example, the multi-precision MAC processor **40** (FIG. **1**), already discussed. The method **300** may generally be implemented in a logic architecture such as, for example, the architecture **180** (FIG. **16**), already discussed. More particularly, the method **300** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality logic hardware using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof.

[0091] Illustrated processing block **302** calculates, via multipliers, signed magnitude values in a first precision and a second precision. Additionally, block **304** calculates/determines a first set of additional partial products in the first precision, wherein block **306** calculates/determines a second set of additional partial products in the second precision.

[0092] FIG. **27** shows another method **310** of operating a performance-enhanced MAC processor such as, for example, the multi-precision MAC processor **40** (FIG. **1**), already discussed. The method **310** may generally be implemented in a logic architecture such as, for example, the architecture **190** (FIG. **17**), already discussed. More particularly, the method **310** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality logic hardware using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof.

[0093] Illustrated processing block **312** sorts individual exponents of floating point representations to identify a largest exponent. Additionally, block **314** denormalizes multiplier products to the largest exponent, wherein block **316** sums the denormalized multiplier products to obtain a product sum. In an embodiment, block **318** normalizes the product sum to a single floating point value.

[0094] FIG. **28** shows another method **320** of operating a performance-enhanced MAC processor such as, for example, the multi-precision MAC processor **40** (FIG. **1**), already discussed. The method **320** may generally be implemented in a logic architecture such as, for example, the architecture **200** (FIG. **18**), already discussed. More particularly, the method **320** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality logic hardware using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof.

[0095] Illustrated processing block **322** denormalizes, at each subsequent arithmetic block, a smaller of two values to a larger value. Additionally, block **324** sums the two values.

[0096] FIG. **29**A shows a method **301** of fabricating a performance-enhanced MAC processor. The method **301** may generally be used to fabricate a multi-precision MAC processor such as, for example, the MAC processor **40** (FIG. **1**), already discussed. More particularly, the method **301** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality logic hardware using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof. The method **301** may also be implemented via suitable semiconductor processes such as, for example, deposition, cutting and/or etching techniques.

[0097] Illustrated processing block **303** provides one or more substrates such as, for example, silicon, sapphire, gallium arsenide, etc. Processing block **305** couples logic (e.g., transistor array and other integrated circuit/IC components) to the substrate(s). In the illustrated example, the logic is implemented at least partly in one or more of configurable or fixed-functionality hardware. Moreover, the logic includes a plurality of arithmetic blocks (e.g., ALUs), wherein the plurality of arithmetic blocks each contain multiple multipliers. Moreover, one or more intermediate multipliers may be of a size that is less than precisions supported by arithmetic blocks containing the one or more intermediate multipliers.

[0098] FIG. **29**B shows another method **311** of operating a performance-enhanced MAC processor such as, for example, the multi-precision MAC processor **40** (FIG. **1**), already discussed. The method **311** may generally be imple-

mented in a logic architecture such as, for example, the architecture **200** (FIG. **18**), already discussed. More particularly, the method **311** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality logic hardware using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof.

[0099] Illustrated processing block **313** combines multipliers one or more of 1) within each arithmetic block, or 2) across multiple arithmetic blocks. Additionally, block **315** may map one or more smaller multipliers to partial products of the intermediate multiplier(s), wherein the smaller multiplier(s) are of a size that is less than the size of the intermediate multiplier(s). In addition, block **317** may combine the intermediate multiplier(s) to obtain one or more larger multipliers, wherein the larger multiplier(s) are of a size that is greater than the size of the intermediate multiplier (s). In an embodiment, block **319** sums partial products in rank order, wherein block **321** shifts the summed partial products to obtain shifted partial products. In such a case, block **323** adds the shifted partial products to obtain one or more of larger multipliers, sums of larger multipliers or sums of smaller multipliers. Moreover, block **325** may pre-code groups of smaller multiplier products, wherein block **327** adds the pre-coded groups of smaller multiplier products.

[0100] FIG. **29**C shows another method **330** of operating a performance-enhanced MAC processor such as, for example, the multi-precision MAC processor **40** (FIG. **1**), already discussed. The method **330** may generally be implemented in a logic architecture such as, for example, the architecture **200** (FIG. **18**), already discussed. More particularly, the method **330** may be implemented in one or more modules as a set of logic instructions stored in a machine- or computer-readable storage medium such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality logic hardware using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof.

[0101] Illustrated processing block **332** sources one or more arithmetic blocks by a plurality of input channels. In an embodiment, block **334** decomposes each of the plurality of input channels into smaller input channels.

[0102] Turning now to FIG. **30**A, a performance-enhanced computing system **340** is shown. The system **340** may generally be part of an electronic device/platform having computing functionality (e.g., personal digital assistant/PDA, notebook computer, tablet computer, convertible tablet, server), communications functionality (e.g., smart phone), imaging functionality (e.g., camera, camcorder), media playing functionality (e.g., smart television/TV), wearable functionality (e.g., watch, eyewear, headwear, footwear, jewelry), vehicular functionality (e.g., car, truck, motorcycle), robotic functionality (e.g., autonomous robot), Internet of Things (IoT) functionality, etc., or any combination thereof. In the illustrated example, the system **340** includes a host processor **342** (e.g., central processing unit/CPU) having an integrated memory controller (IMC) **344** that is coupled to a system memory **346**.

[0103] The illustrated system **340** also includes an input output (IO) module **348** implemented together with the host processor **342**, an AI accelerator **351** (e.g., DNN processing engine) and a graphics processor **350** (e.g., graphics processing unit/GPU) on a semiconductor die **352** as a system on chip (SoC). The illustrated IO module **348** communicates with, for example, a display **354** (e.g., touch screen, liquid crystal display/LCD, light emitting diode/LED display), a network controller **356** (e.g., wired and/or wireless), and mass storage **358** (e.g., hard disk drive/HDD, optical disk, solid state drive/SSD, flash memory).

[0104] In an embodiment, the AI accelerator **351** includes a multi-precision MAC processor such as, for example, the MAC processor **40** (FIG. **1**), already discussed. Thus, the AI accelerator **351** includes logic (e.g., logic instructions, configurable logic, fixed-functionality hardware logic, etc., or any combination thereof) having a plurality of arithmetic blocks to perform one or more aspects of the method **240** (FIG. **22**), the method **250** (FIG. **23**), the method **260** (FIG. **24**), the method **270** (FIG. **25**A), the method **280** (FIG. **25**B), the method **290** (FIG. **26**A), the method **300** (FIG. **26**B), the method **310** (FIG. **27**) and/or the method **320** (FIG. **28**), already discussed. In an embodiment, the plurality of arithmetic blocks share a single multiplier size that is uniform across the arithmetic blocks, wherein the single multiplier size is less than a maximum precision size supported by the arithmetic blocks. The computing system is therefore considered performance-enhanced at least to the extent that single multiplier size renders the MAC processor more area and/or energy efficient.

[0105] FIG. **30**B shows a semiconductor package apparatus **360**. The illustrated apparatus **360** includes one or more substrates **362** (e.g., silicon, sapphire, gallium arsenide) and logic **364** (e.g., transistor array and other integrated circuit/IC components) coupled to the substrate(s) **362**. The logic **364** may be implemented at least partly in configurable logic or fixed-functionality logic hardware. In one example, the logic **364** implements one or more aspects of the method **240** (FIG. **22**), the method **250** (FIG. **23**), the method **260** (FIG. **24**), the method **270** (FIG. **25**A), the method **280** (FIG. **25**B), the method **290** (FIG. **26**A), the method **300** (FIG. **26**B), the method **310** (FIG. **27**) and/or the method **320** (FIG. **28**), already discussed. In an embodiment, the logic **364** includes a plurality of arithmetic blocks that share a single multiplier size, wherein the single multiplier size is uniform across the arithmetic blocks, and wherein the single multiplier size is less than a maximum precision size supported by the arithmetic blocks. The apparatus **360** is therefore considered performance-enhanced at least to the extent that single multiplier size renders the MAC processor more area and/or energy efficient.

[0106] In one example, the logic **364** includes transistor channel regions that are positioned (e.g., embedded) within the substrate(s) **362**. Thus, the interface between the logic **364** and the substrate(s) **172** may not be an abrupt junction. The logic **364** may also be considered to include an epitaxial layer that is grown on an initial wafer of the substrate(s) **362**.

[0107] FIG. **31** illustrates a processor core **400** according to one embodiment. The processor core **400** may be the core for any type of processor, such as a micro-processor, an embedded processor, a digital signal processor (DSP), a network processor, or other device to execute code. Although only one processor core **400** is illustrated in FIG. **31**, a processing element may alternatively include more than one of the processor core **400** illustrated in FIG. **31**. The processor core **400** may be a single-threaded core or, for at

least one embodiment, the processor core **400** may be multithreaded in that it may include more than one hardware thread context (or "logical processor") per core.

[0108] FIG. **31** also illustrates a memory **470** coupled to the processor core **400**. The memory **470** may be any of a wide variety of memories (including various layers of memory hierarchy) as are known or otherwise available to those of skill in the art. The memory **470** may include one or more code **413** instruction(s) to be executed by the processor core **400**, wherein the code **413** may implement one or more aspects of the method **240** (FIG. **22**), the method **250** (FIG. **23**), the method **260** (FIG. **24**), the method **270** (FIG. **25**A), the method **280** (FIG. **25**B), the method **290** (FIG. **26**A), the method **300** (FIG. **26**B), the method **310** (FIG. **27**) and/or the method **320** (FIG. **28**), already discussed. The processor core **400** follows a program sequence of instructions indicated by the code **413**. Each instruction may enter a front end portion **410** and be processed by one or more decoders **420**. The decoder **420** may generate as its output a micro operation such as a fixed width micro operation in a predefined format, or may generate other instructions, microinstructions, or control signals which reflect the original code instruction. The illustrated front end portion **410** also includes register renaming logic **425** and scheduling logic **430**, which generally allocate resources and queue the operation corresponding to the convert instruction for execution.

[0109] The processor core **400** is shown including execution logic **450** having a set of execution units **455-1** through **455**-N. Some embodiments may include a number of execution units dedicated to specific functions or sets of functions. Other embodiments may include only one execution unit or one execution unit that can perform a particular function. The illustrated execution logic **450** performs the operations specified by code instructions.

[0110] After completion of execution of the operations specified by the code instructions, back end logic **460** retires the instructions of the code **413**. In one embodiment, the processor core **400** allows out of order execution but requires in order retirement of instructions. Retirement logic **465** may take a variety of forms as known to those of skill in the art (e.g., re-order buffers or the like). In this manner, the processor core **400** is transformed during execution of the code **413**, at least in terms of the output generated by the decoder, the hardware registers and tables utilized by the register renaming logic **425**, and any registers (not shown) modified by the execution logic **450**.

[0111] Although not illustrated in FIG. **31**, a processing element may include other elements on chip with the processor core **400**. For example, a processing element may include memory control logic along with the processor core **400**. The processing element may include I/O control logic and/or may include I/O control logic integrated with memory control logic. The processing element may also include one or more caches.

[0112] Referring now to FIG. **32**, shown is a block diagram of a computing system **1000** embodiment in accordance with an embodiment. Shown in FIG. **32** is a multiprocessor system **1000** that includes a first processing element **1070** and a second processing element **1080**. While two processing elements **1070** and **1080** are shown, it is to be understood that an embodiment of the system **1000** may also include only one such processing element.

[0113] The system **1000** is illustrated as a point-to-point interconnect system, wherein the first processing element **1070** and the second processing element **1080** are coupled via a point-to-point interconnect **1050**. It should be understood that any or all of the interconnects illustrated in FIG. **32** may be implemented as a multi-drop bus rather than point-to-point interconnect.

[0114] As shown in FIG. **32**, each of processing elements **1070** and **1080** may be multicore processors, including first and second processor cores (i.e., processor cores **1074***a* and **1074***b* and processor cores **1084***a* and **1084***b*). Such cores **1074***a*, **1074***b*, **1084***a*, **1084***b* may be configured to execute instruction code in a manner similar to that discussed above in connection with FIG. **31**.

[0115] Each processing element **1070**, **1080** may include at least one shared cache **1896***a*, **1896***b*. The shared cache **1896***a*, **1896***b* may store data (e.g., instructions) that are utilized by one or more components of the processor, such as the cores **1074***a*, **1074***b* and **1084***a*, **1084***b*, respectively. For example, the shared cache **1896***a*, **1896***b* may locally cache data stored in a memory **1032**, **1034** for faster access by components of the processor. In one or more embodiments, the shared cache **1896***a*, **1896***b* may include one or more mid-level caches, such as level 2 (L2), level 3 (L3), level 4 (L4), or other levels of cache, a last level cache (LLC), and/or combinations thereof.

[0116] While shown with only two processing elements **1070**, **1080**, it is to be understood that the scope of the embodiments are not so limited. In other embodiments, one or more additional processing elements may be present in a given processor. Alternatively, one or more of processing elements **1070**, **1080** may be an element other than a processor, such as an accelerator or a field programmable gate array. For example, additional processing element(s) may include additional processors(s) that are the same as a first processor **1070**, additional processor(s) that are heterogeneous or asymmetric to processor a first processor **1070**, accelerators (such as, e.g., graphics accelerators or digital signal processing (DSP) units), field programmable gate arrays, or any other processing element. There can be a variety of differences between the processing elements **1070**, **1080** in terms of a spectrum of metrics of merit including architectural, micro architectural, thermal, power consumption characteristics, and the like. These differences may effectively manifest themselves as asymmetry and heterogeneity amongst the processing elements **1070**, **1080**. For at least one embodiment, the various processing elements **1070**, **1080** may reside in the same die package.

[0117] The first processing element **1070** may further include memory controller logic (MC) **1072** and point-to-point (P-P) interfaces **1076** and **1078**. Similarly, the second processing element **1080** may include a MC **1082** and P-P interfaces **1086** and **1088**. As shown in FIG. **32**, MC's **1072** and **1082** couple the processors to respective memories, namely a memory **1032** and a memory **1034**, which may be portions of main memory locally attached to the respective processors. While the MC **1072** and **1082** is illustrated as integrated into the processing elements **1070**, **1080**, for alternative embodiments the MC logic may be discrete logic outside the processing elements **1070**, **1080** rather than integrated therein.

[0118] The first processing element **1070** and the second processing element **1080** may be coupled to an I/O subsystem **1090** via P-P interconnects **1076** **1086**, respectively. As

shown in FIG. **32**, the I/O subsystem **1090** includes P-P interfaces **1094** and **1098**. Furthermore, I/O subsystem **1090** includes an interface **1092** to couple I/O subsystem **1090** with a high performance graphics engine **1038**. In one embodiment, bus **1049** may be used to couple the graphics engine **1038** to the I/O subsystem **1090**. Alternately, a point-to-point interconnect may couple these components.

[0119] In turn, I/O subsystem **1090** may be coupled to a first bus **1016** via an interface **1096**. In one embodiment, the first bus **1016** may be a Peripheral Component Interconnect (PCI) bus, or a bus such as a PCI Express bus or another third generation I/O interconnect bus, although the scope of the embodiments are not so limited.

[0120] As shown in FIG. **32**, various I/O devices **1014** (e.g., biometric scanners, speakers, cameras, sensors) may be coupled to the first bus **1016**, along with a bus bridge **1018** which may couple the first bus **1016** to a second bus **1020**. In one embodiment, the second bus **1020** may be a low pin count (LPC) bus. Various devices may be coupled to the second bus **1020** including, for example, a keyboard/mouse **1012**, communication device(s) **1026**, and a data storage unit **1019** such as a disk drive or other mass storage device which may include code **1030**, in one embodiment. The illustrated code **1030** may implement one or more aspects of the method **240** (FIG. **22**), the method **250** (FIG. **23**), the method **260** (FIG. **24**), the method **270** (FIG. **25**A), the method **280** (FIG. **25**B), the method **290** (FIG. **26**A), the method **300** (FIG. **26**B), the method **310** (FIG. **27**) and/or the method **320** (FIG. **28**), already discussed. Further, an audio I/O **1024** may be coupled to second bus **1020** and a battery **1010** may supply power to the computing system **1000**.

[0121] Note that other embodiments are contemplated. For example, instead of the point-to-point architecture of FIG. **32**, a system may implement a multi-drop bus or another such communication topology. Also, the elements of FIG. **32** may alternatively be partitioned using more or fewer integrated chips than shown in FIG. **32**.

ADDITIONAL NOTES AND EXAMPLES

[0122] Example 1 includes a multiply-accumulate (MAC) processor comprising one or more substrates, and logic coupled to the one or more substrates, wherein the logic is implemented at least partly in one or more of configurable or fixed-functionality hardware, the logic including a plurality of arithmetic blocks, wherein the plurality of arithmetic blocks each contain multiple multipliers, and wherein the logic is to combine multipliers one or more of within each arithmetic block or across multiple arithmetic blocks.

[0123] Example 2 includes the MAC processor of Example 1, wherein one or more intermediate multipliers are of a size that is less than precisions supported by arithmetic blocks containing the one or more intermediate multipliers.

[0124] Example 3 includes the MAC processor of Example 2, wherein the logic is to map one or more smaller multipliers to partial products of the one or more intermediate multipliers, and wherein the one or more smaller multipliers are of a size that is less than the size of the one or more intermediate multipliers.

[0125] Example 4 includes the MAC processor of Example 2, wherein the logic is to combine the one or more intermediate multipliers to obtain one or more larger multipliers, and wherein the one or more larger multipliers are of a size that is greater than the size of the one or more intermediate multipliers.

[0126] Example 5 includes the MAC processor of Example 2, wherein the logic is to sum partial products in rank order, and shift the summed partial products to obtain shifted partial products, and add the shifted partial products to obtain one or more of larger multipliers, sums of larger multipliers or sums of smaller multipliers.

[0127] Example 6 includes the MAC processor of Example 2, wherein the logic is to pre-code groups of smaller multiplier products, and add the pre-coded groups of smaller multiplier products.

[0128] Example 7 includes the MAC processor of Example 6, wherein the logic is to multiply pre-coded combinations of smaller multiplier products by a constant to obtain a sum.

[0129] Example 8 includes the MAC processor of Example 1, wherein all of the multiple multipliers are of a same precision.

[0130] Example 9 includes the MAC processor of Example 1, wherein the logic is to source one or more arithmetic blocks by a plurality of input channels, and decompose each of the plurality of input channels into smaller input channels.

[0131] Example 10 includes the MAC processor of Example 1, wherein the logic is to add multiplier outputs in rank order across the plurality of arithmetic blocks.

[0132] Example 11 includes the MAC processor of Example 1, wherein the logic is to decode subsets of weights and activations as a multiplier pre-process operation.

[0133] Example 12 includes the MAC processor of Example 1, wherein the logic is to invert individual partial products to operate one or more multipliers as a signed magnitude multiplier.

[0134] Example 13 includes the MAC processor of Example 12, wherein the logic is to add a single mixed radix partial product, and wherein a final partial product of a lower radix operates as a subset of possibilities of a higher radix.

[0135] Example 14 includes the MAC processor of Example 12, wherein, for a group of multipliers, the logic is to sum ranks of partial products, and sum a group of partial products in a different radix separately from the ranks of partial products.

[0136] Example 15 includes the MAC processor of Example 14, wherein the group of multipliers one or more of provide unsigned multiplication or are in signed magnitude format.

[0137] Example 16 includes the MAC processor of Example 1, wherein the logic is to zero out a top portion of partial products, zero out a bottom portion of the partial products, compress ranks of each set of original partial products independently, and shift groups of ranks into an alignment of a smaller precision.

[0138] Example 17 includes the MAC processor of Example 16, wherein the logic is to calculate, via multipliers, signed magnitude values in a first precision and a second precision, calculate a first set of additional partial products in the first precision, and calculate a second set of additional partial products in the second precision.

[0139] Example 18 includes the MAC processor of Example 1, wherein the logic is to sort individual exponents of floating point representations to identify a largest exponent, denormalize multiplier products to the largest exponent, sum the denormalized multiplier products to obtain a product sum, and normalize the product sum to a single floating point value.

[0140] Example 19 includes the MAC processor of Example 1, wherein the plurality of arithmetic blocks are cascaded in a sequence, and wherein the logic is to denormalize, at each subsequent arithmetic block, a smaller of two values to a larger value, and sum the two values.

[0141] Example 20 includes the MAC processor of any one of Examples 1 to 19, wherein the logic is to arrange sparsity information for activations and weights in accordance with a bitmap format that is common to multiple precisions.

[0142] Example 21 includes the MAC processor of any one of Examples 1 to 19, wherein the logic coupled to the one or more substrates includes transistor channel regions that are positioned within the one or more substrates.

[0143] Example 22 includes a computing system comprising a network controller, and a multiply-accumulate (MAC) processor coupled to the network controller, wherein the MAC processor includes logic coupled to one or more substrates, wherein the logic includes a plurality of arithmetic blocks, wherein the plurality of arithmetic blocks each contain multiple multipliers, and wherein the logic is to combine multipliers one or more of within each arithmetic block or across multiple arithmetic blocks.

[0144] Example 23 includes the computing system of Example 22, wherein one or more intermediate multipliers are of a size that is less than precisions supported by arithmetic blocks containing the one or more intermediate multipliers.

[0145] Example 24 includes a method comprising providing one or more substrates, and coupling logic to the one or more substrates, wherein the logic is implemented at least partly in one or more of configurable or fixed-functionality hardware, the logic including a plurality of arithmetic blocks, wherein the plurality of arithmetic blocks each contain multiple multipliers, and wherein the logic is to combine multipliers one or more of within each arithmetic block or across multiple arithmetic blocks.

[0146] Example 25 includes the method of Example 24, wherein one or more intermediate multipliers are of a size that is less than precisions supported by arithmetic blocks containing the one or more intermediate multipliers.

[0147] Example 26 includes means for performing the method of any one of Examples 24 to 25.

[0148] Technology described herein therefore delivers high performance at a fraction of the area and energy costs in DNN accelerators, which may be key to efficient edge inference for various AI applications including imaging, video, and speech applications. The technology also provides a design that is high performance, has low silicon footprint and energy consumption, and can provide a unique edge in terms of better performance, and taking advantages of transistor scaling.

[0149] Embodiments are applicable for use with all types of semiconductor integrated circuit ("IC") chips. Examples of these IC chips include but are not limited to processors, controllers, chipset components, programmable logic arrays (PLAs), memory chips, network chips, systems on chip (SoCs), SSD/NAND controller ASICs, and the like. In addition, in some of the drawings, signal conductor lines are represented with lines. Some may be different, to indicate more constituent signal paths, have a number label, to indicate a number of constituent signal paths, and/or have arrows at one or more ends, to indicate primary information flow direction. This, however, should not be construed in a limiting manner. Rather, such added detail may be used in connection with one or more exemplary embodiments to facilitate easier understanding of a circuit. Any represented signal lines, whether or not having additional information, may actually comprise one or more signals that may travel in multiple directions and may be implemented with any suitable type of signal scheme, e.g., digital or analog lines implemented with differential pairs, optical fiber lines, and/or single-ended lines.

[0150] Example sizes/models/values/ranges may have been given, although embodiments are not limited to the same. As manufacturing techniques (e.g., photolithography) mature over time, it is expected that devices of smaller size could be manufactured. In addition, well known power/ground connections to IC chips and other components may or may not be shown within the figures, for simplicity of illustration and discussion, and so as not to obscure certain aspects of the embodiments. Further, arrangements may be shown in block diagram form in order to avoid obscuring embodiments, and also in view of the fact that specifics with respect to implementation of such block diagram arrangements are highly dependent upon the computing system within which the embodiment is to be implemented, i.e., such specifics should be well within purview of one skilled in the art. Where specific details (e.g., circuits) are set forth in order to describe example embodiments, it should be apparent to one skilled in the art that embodiments can be practiced without, or with variation of, these specific details. The description is thus to be regarded as illustrative instead of limiting.

[0151] The term "coupled" may be used herein to refer to any type of relationship, direct or indirect, between the components in question, and may apply to electrical, mechanical, fluid, optical, electromagnetic, electromechanical or other connections. In addition, the terms "first", "second", etc. may be used herein only to facilitate discussion, and carry no particular temporal or chronological significance unless otherwise indicated.

[0152] As used in this application and in the claims, a list of items joined by the term "one or more of" may mean any combination of the listed terms. For example, the phrases "one or more of A, B or C" may mean A; B; C; A and B; A and C; B and C; or A, B and C.

[0153] Those skilled in the art will appreciate from the foregoing description that the broad techniques of the embodiments can be implemented in a variety of forms. Therefore, while the embodiments have been described in connection with particular examples thereof, the true scope of the embodiments should not be so limited since other modifications will become apparent to the skilled practitioner upon a study of the drawings, specification, and following claims.

We claim:

1. A multiply-accumulate (MAC) processor comprising:

one or more substrates; and

logic coupled to the one or more substrates, wherein the logic is implemented at least partly in one or more of configurable or fixed-functionality hardware, the logic including a plurality of arithmetic blocks, wherein the plurality of arithmetic blocks each contain multiple multipliers, and wherein the logic is to combine multipliers either within each arithmetic block or across multiple arithmetic blocks.

2. The MAC processor of claim 1, wherein one or more intermediate multipliers are of a size that is less than precisions supported by arithmetic blocks containing the one or more intermediate multipliers.

3. The MAC processor of claim 2, wherein the logic is to map one or more smaller multipliers to partial products of the one or more intermediate multipliers, and wherein the one or more smaller multipliers are of a size that is less than the size of the one or more intermediate multipliers.

4. The MAC processor of claim 2, wherein the logic is to combine the one or more intermediate multipliers to obtain one or more larger multipliers, and wherein the one or more larger multipliers are of a size that is greater than the size of the one or more intermediate multipliers.

5. The MAC processor of claim 2, wherein the logic is to:
sum partial products in rank order; and
shift the summed partial products to obtain shifted partial products; and
add the shifted partial products to obtain one or more of larger multipliers, sums of larger multipliers or sums of smaller multipliers.

6. The MAC processor of claim 2, wherein the logic is to:
pre-code groups of smaller multiplier products; and
add the pre-coded groups of smaller multiplier products.

7. The MAC processor of claim 6, wherein the logic is to multiply pre-coded combinations of smaller multiplier products by a constant to obtain a sum.

8. The MAC processor of claim 1, wherein all of the multiple multipliers are of a same precision.

9. The MAC processor of claim 1, wherein the logic is to:
source one or more arithmetic blocks by a plurality of input channels; and
decompose each of the plurality of input channels into smaller input channels.

10. The MAC processor of claim 1, wherein the logic is to add multiplier outputs in rank order across the plurality of arithmetic blocks.

11. The MAC processor of claim 1, wherein the logic is to decode subsets of weights and activations as a multiplier pre-process operation.

12. The MAC processor of claim 1, wherein the logic is to invert individual partial products to operate one or more multipliers as a signed magnitude multiplier.

13. The MAC processor of claim 12, wherein the logic is to add a single mixed radix partial product, and wherein a final partial product of a lower radix operates as a subset of possibilities of a higher radix.

14. The MAC processor of claim 12, wherein, for a group of multipliers, the logic is to:
sum ranks of partial products; and
sum a group of partial products in a different radix separately from the ranks of partial products.

15. The MAC processor of claim 14, wherein the group of multipliers one or more of provide unsigned multiplication or are in signed magnitude format.

16. The MAC processor of claim 1, wherein the logic is to:
zero out a top portion of partial products;
zero out a bottom portion of the partial products;
compress ranks of each set of original partial products independently; and
shift groups of ranks into an alignment of a smaller precision.

17. The MAC processor of claim 16, wherein the logic is to:
calculate, via multipliers, signed magnitude values in a first precision and a second precision;
calculate a first set of additional partial products in the first precision; and
calculate a second set of additional partial products in the second precision.

18. The MAC processor of claim 1, wherein the logic is to:
sort individual exponents of floating point representations to identify a largest exponent;
denormalize multiplier products to the largest exponent;
sum the denormalized multiplier products to obtain a product sum; and
normalize the product sum to a single floating point value.

19. The MAC processor of claim 1, wherein the plurality of arithmetic blocks are cascaded in a sequence, and wherein the logic is to:
denormalize, at each subsequent arithmetic block, a smaller of two values to a larger value; and
sum the two values.

20. The MAC processor of claim 1, wherein the logic is to arrange sparsity information for activations and weights in accordance with a bitmap format that is common to multiple precisions.

21. The MAC processor of claim 1, wherein the logic coupled to the one or more substrates includes transistor channel regions that are positioned within the one or more substrates.

22. A computing system comprising:
a network controller; and
a multiply-accumulate (MAC) processor coupled to the network controller, wherein the MAC processor includes logic coupled to one or more substrates, wherein the logic includes a plurality of arithmetic blocks, wherein the plurality of arithmetic blocks each contain multiple multipliers, and wherein the logic is to combine multipliers either within each arithmetic block or across multiple arithmetic blocks.

23. The computing system of claim 22, wherein one or more intermediate multipliers are of a size that is less than precisions supported by arithmetic blocks containing the one or more intermediate multipliers.

24. A method comprising:
providing one or more substrates; and
coupling logic to the one or more substrates, wherein the logic is implemented at least partly in one or more of configurable or fixed-functionality hardware, the logic including a plurality of arithmetic blocks, wherein the plurality of arithmetic blocks each contain multiple multipliers, and wherein the logic is to combine multipliers either within each arithmetic block or across multiple arithmetic blocks.

25. The method of claim 24, wherein one or more intermediate multipliers are of a size that is less than precisions supported by arithmetic blocks containing the one or more intermediate multipliers.

* * * * *